HW Portfolio

최 세 현

목차

교육/프로젝트

1. FPGA-Based Simple MIPS Processor

개인/프로젝트

1. 공정 미세화 한계 극복을 위한 AXI 기반 SoC 통합 및 PIM 개념의 Near-Memory 하드웨어 가속기 설계

교육/프로젝트

PROJECT 01. FPGA-Based Simple MIPS Processor

수행기간

2026.01

수행인원

1명

사용기술

VIVADO(verilog), FPGA

수행내용

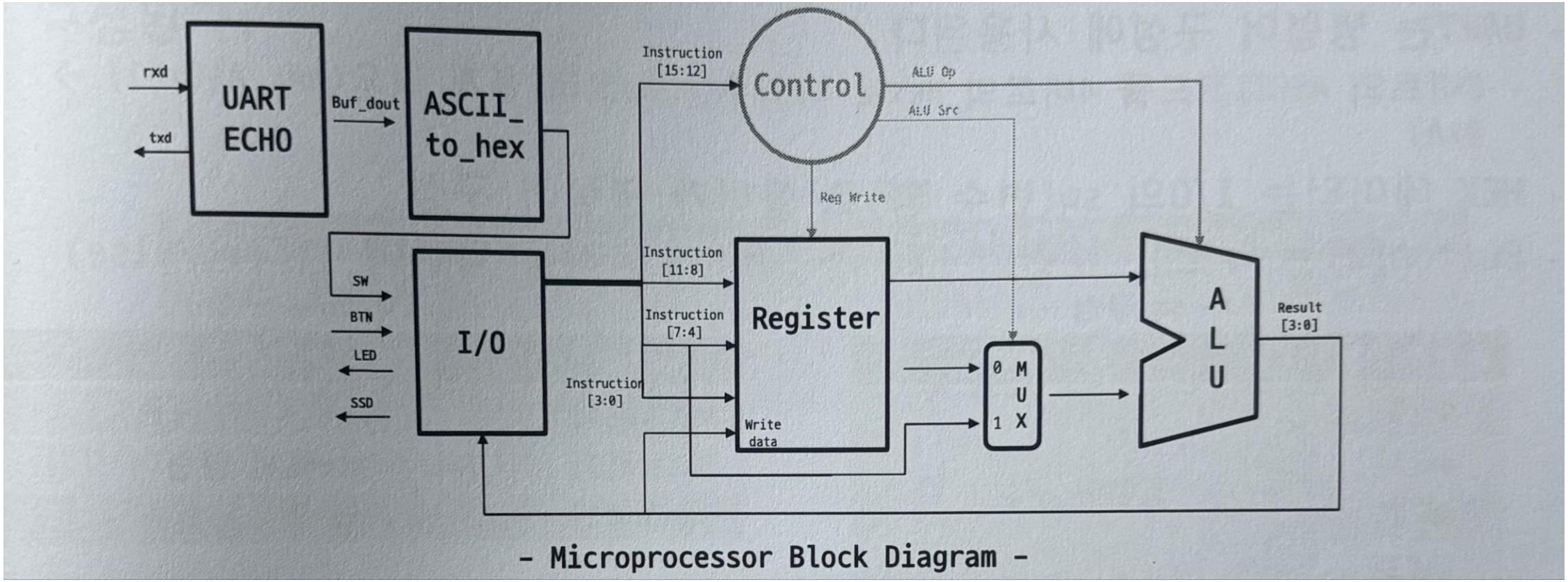
(1) 기본적인 MIPS 명령어 셋을 처리할 수 있는 데이터패스 및 제어 유닛 설계

(2) ALU 내부에 CLA를 직접 설계 및 적용하여, 기존 RCA 대비 덧셈 연산 지연시간 개선

(3) UART 통신 프로토콜을 구현하여, PC로부터 직접 연산용 데이터를 입력받아 처리하는 I/O 환경 구축

(4) FSM 기반의 상태 제어 로직을 구현하고, FPGA 보드의 푸시 버튼을 활용한 사이클 단위 디버깅 및 동작 검증 완료

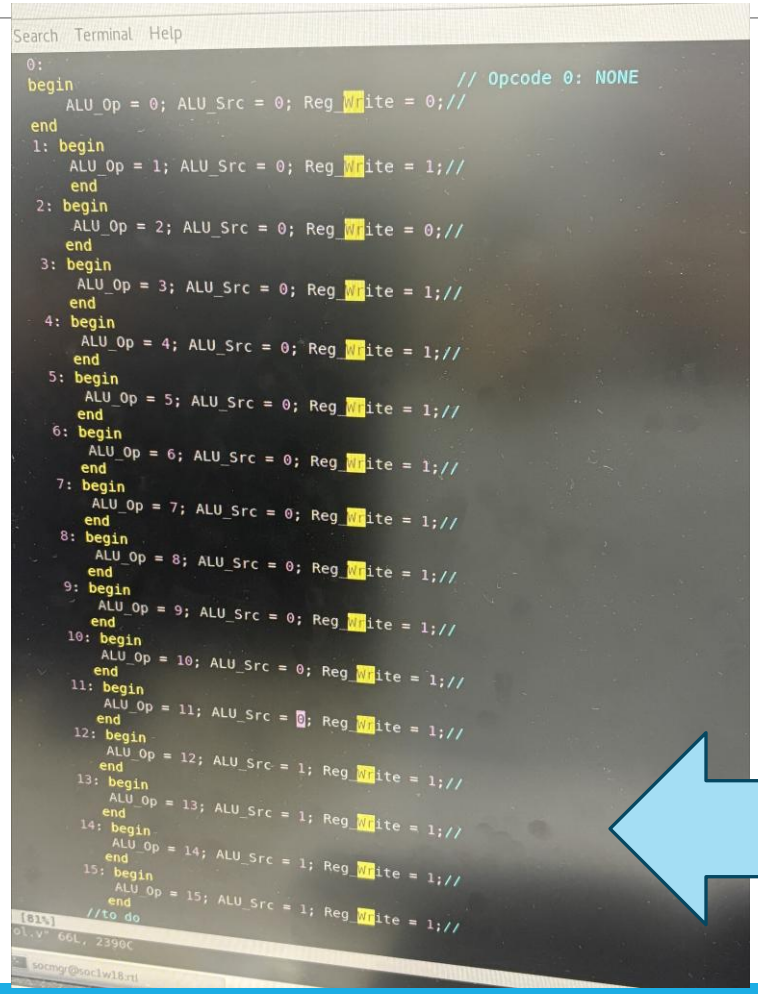
설계 구조



- Microprocessor Block Diagram -

FPGA-Based Simple MIPS Processor

Control 모듈에서 각 opcode의 값에 따른 output값 설정



```
Search  Terminal  Help
0:                                          // Opcode 0: NONE
begin
    ALU_Op = 0; ALU_Src = 0; Reg_Write = 0;//
end
1: begin
    ALU_Op = 1; ALU_Src = 0; Reg_Write = 1;//
    end
2: begin
    ALU_Op = 2; ALU_Src = 0; Reg_Write = 0;//
    end
3: begin
    ALU_Op = 3; ALU_Src = 0; Reg_Write = 1;//
    end
4: begin
    ALU_Op = 4; ALU_Src = 0; Reg_Write = 1;//
    end
5: begin
    ALU_Op = 5; ALU_Src = 0; Reg_Write = 1;//
    end
6: begin
    ALU_Op = 6; ALU_Src = 0; Reg_Write = 1;//
    end
7: begin
    ALU_Op = 7; ALU_Src = 0; Reg_Write = 1;//
    end
8: begin
    ALU_Op = 8; ALU_Src = 0; Reg_Write = 1;//
    end
9: begin
    ALU_Op = 9; ALU_Src = 0; Reg_Write = 1;//
    end
10: begin
    ALU_Op = 10; ALU_Src = 0; Reg_Write = 1;//
    end
11: begin
    ALU_Op = 11; ALU_Src = 0; Reg_Write = 1;//
    end
12: begin
    ALU_Op = 12; ALU_Src = 1; Reg_Write = 1;//
    end
13: begin
    ALU_Op = 13; ALU_Src = 1; Reg_Write = 1;//
    end
14: begin
    ALU_Op = 14; ALU_Src = 1; Reg_Write = 1;//
    end
15: begin
    ALU_Op = 15; ALU_Src = 1; Reg_Write = 1;//
    end
//to do
[81%]
ol.v" 66L, 2390C
socmg@soc1w18:rtl
```

Opcode 12~15는 레지스터가 아닌 외부(pc)의 입력을 받아 사용하므로 ALU_Src = 1

Final Project Milestone1

Testbench 수정 및 검증

순서대로

ADD

SUB

WRITE

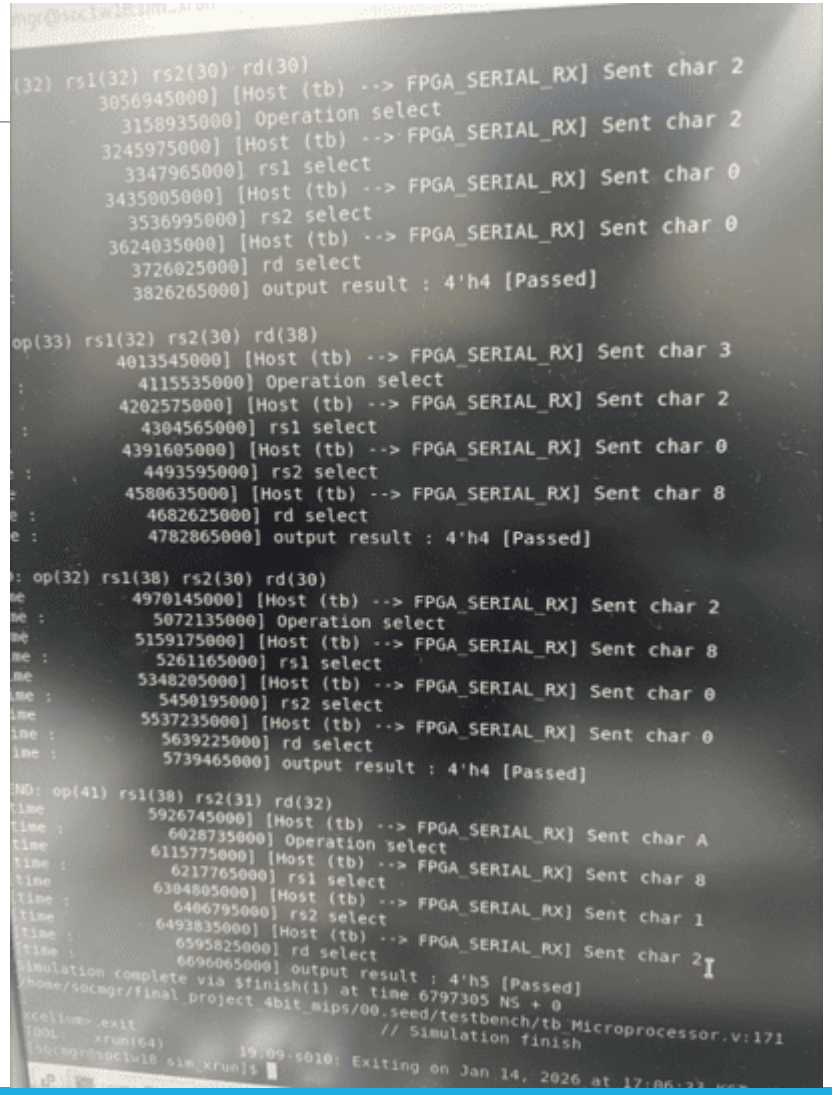
READ

COPY

READ

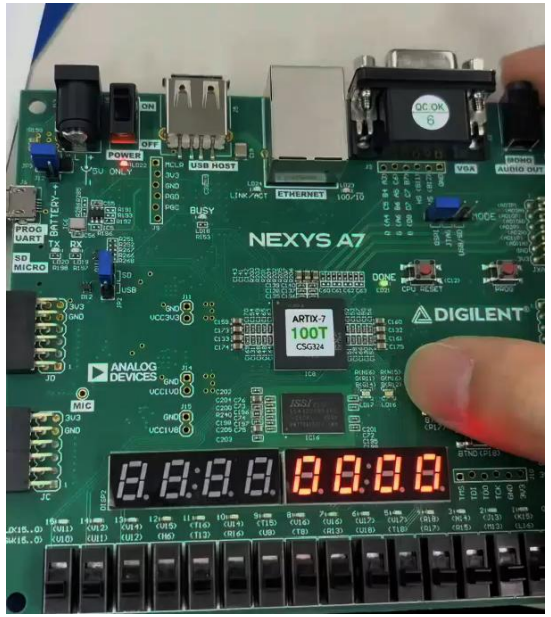
ADD

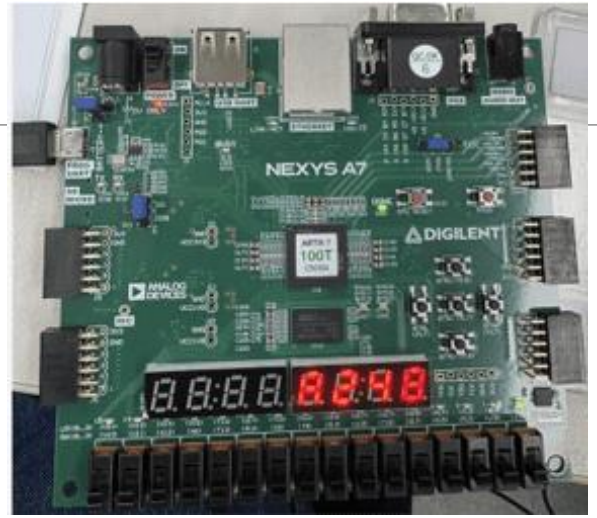
동작

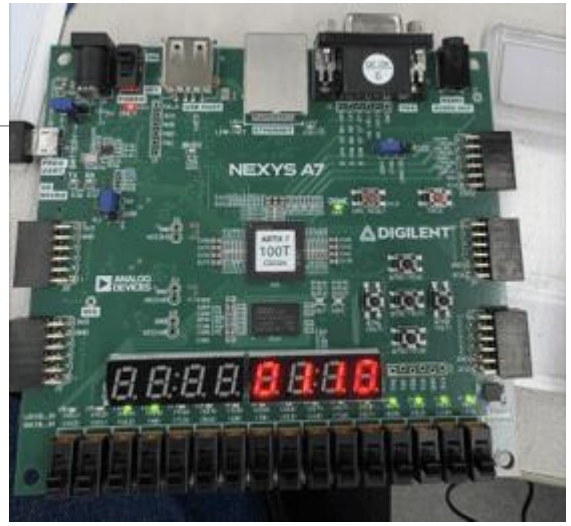


Final Project Milestone1

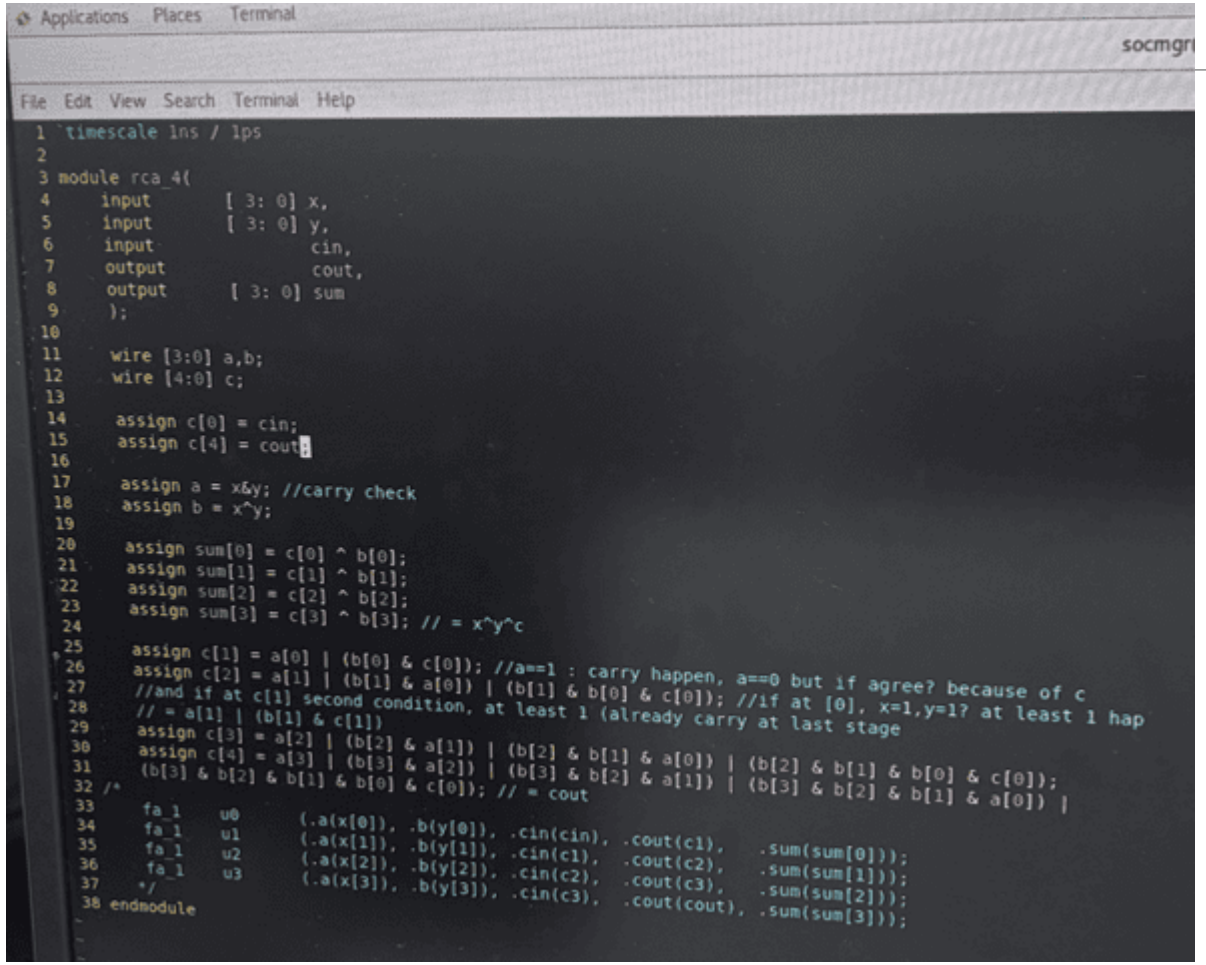
- A248 → B84A
- 시뮬레이션의 결과를 FPGA에서 확인







Final Project Milestone2 - CLA : 병렬 carry 계산, 이전 carry직접 사용x



```
`timescale 1ns / 1ps

module rca_4(
    input       [ 3: 0] x,
    input       [ 3: 0] y,
    input               cin,
    output              cout,
    output      [ 3: 0] sum
    );

    wire [3:0] a,b;
    wire [4:0] c;

    assign c[0] = cin;
    assign c[4] = cout;

    assign a = x&y; //carry check
    assign b = x^y;

    assign sum[0] = c[0] ^ b[0];
    assign sum[1] = c[1] ^ b[1];
    assign sum[2] = c[2] ^ b[2];
    assign sum[3] = c[3] ^ b[3]; // = x^y^c

    assign c[1] = a[0] | (b[0] & c[0]); //a==1 : carry happen, a==0 but if agree? because of c
    assign c[2] = a[1] | (b[1] & a[0]) | (b[1] & b[0] & c[0]); //if at [0], x=1,y=1? at least 1 hap
    //and if at c[1] second condition, at least 1 (already carry at last stage
    // = a[1] | (b[1] & c[1])
    assign c[3] = a[2] | (b[2] & a[1]) | (b[2] & b[1] & a[0]) | (b[2] & b[1] & b[0] & c[0]);
    assign c[4] = a[3] | (b[3] & a[2]) | (b[3] & b[2] & a[1]) | (b[3] & b[2] & b[1] & a[0]) |
    (b[3] & b[2] & b[1] & b[0] & c[0]); // = cout
/*
    fa_1    u0      (.a(x[0]), .b(y[0]), .cin(cin), .cout(c1),   .sum(sum[0]));
    fa_1    u1      (.a(x[1]), .b(y[1]), .cin(c1),  .cout(c2),   .sum(sum[1]));
    fa_1    u2      (.a(x[2]), .b(y[2]), .cin(c2),  .cout(c3),   .sum(sum[2]));
    fa_1    u3      (.a(x[3]), .b(y[3]), .cin(c3),  .cout(cout), .sum(sum[3]));
   */
endmodule
```

x	y	c	sum	Carry(out)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

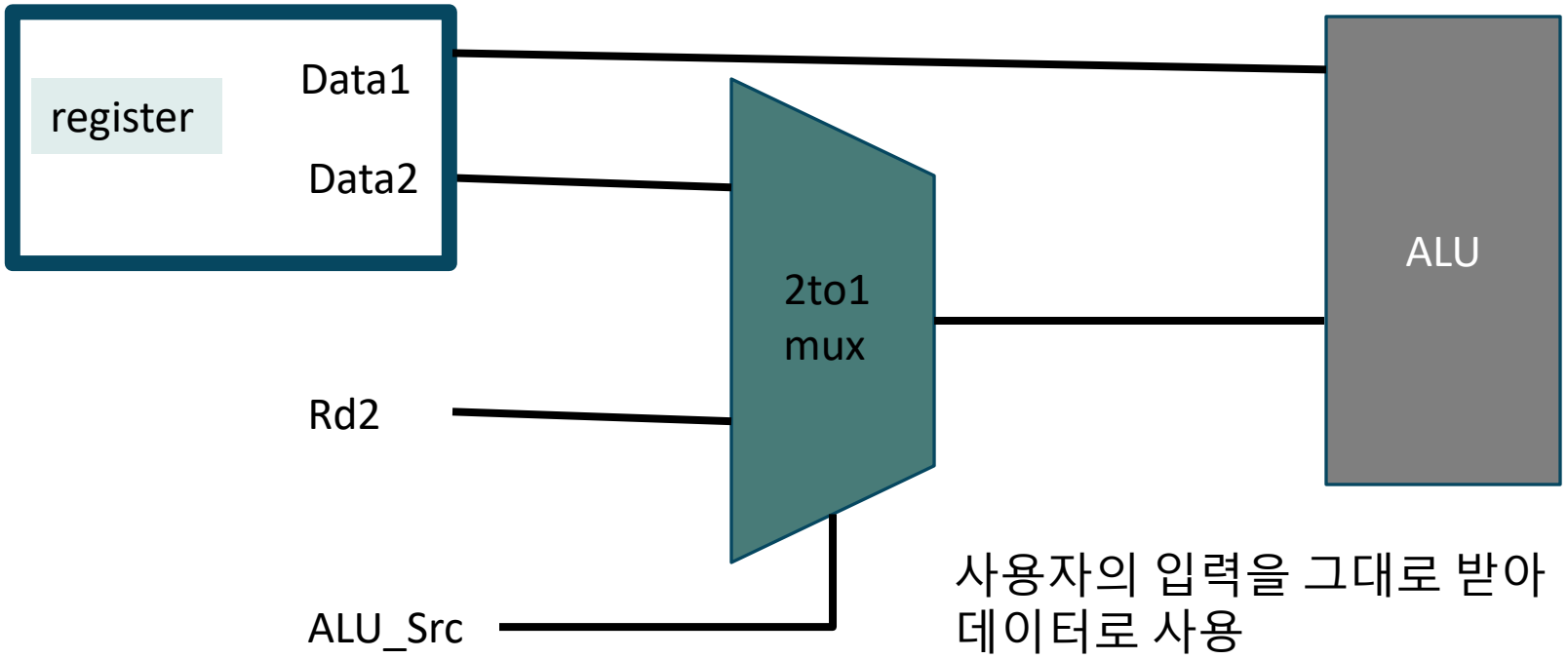
- Sum = x,y,c의 xor
- Carry out = xy+c(x^y)
- C[4] = 최종 cout
- 치환사용

Final Project Milestone3

socmgr@soc1w18:rtl

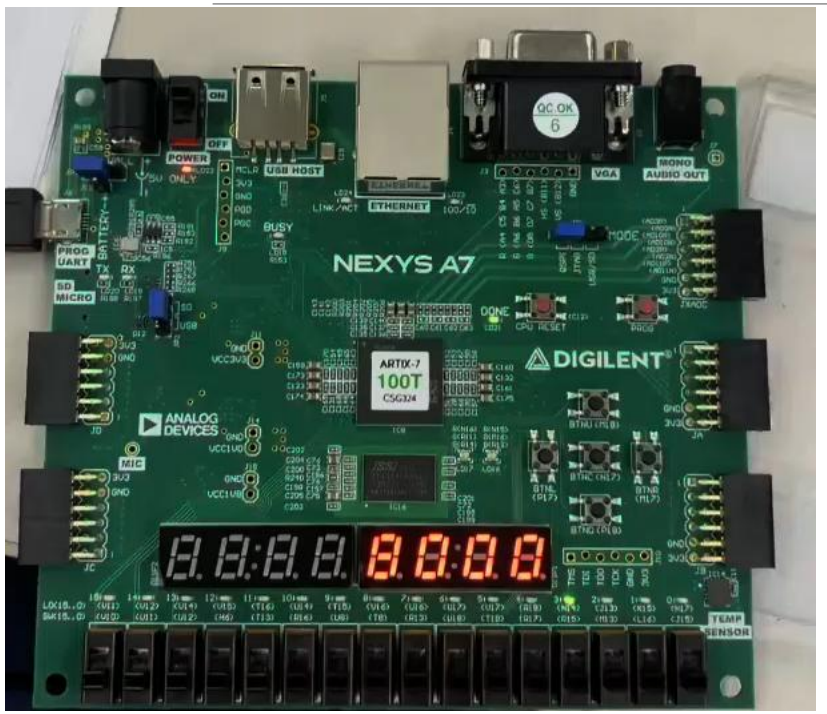
```
1  `timescale 1ns/1ps
2
3  module mux(a,b,sel,o);
4
5  input [3:0] a,b;
6  input sel;
7  output reg [3:0]o;
8
9  always @ (*) begin
10 if(sel==0)
11     o = a;
12 else
13     o = b;
14 end
15 endmodule
```

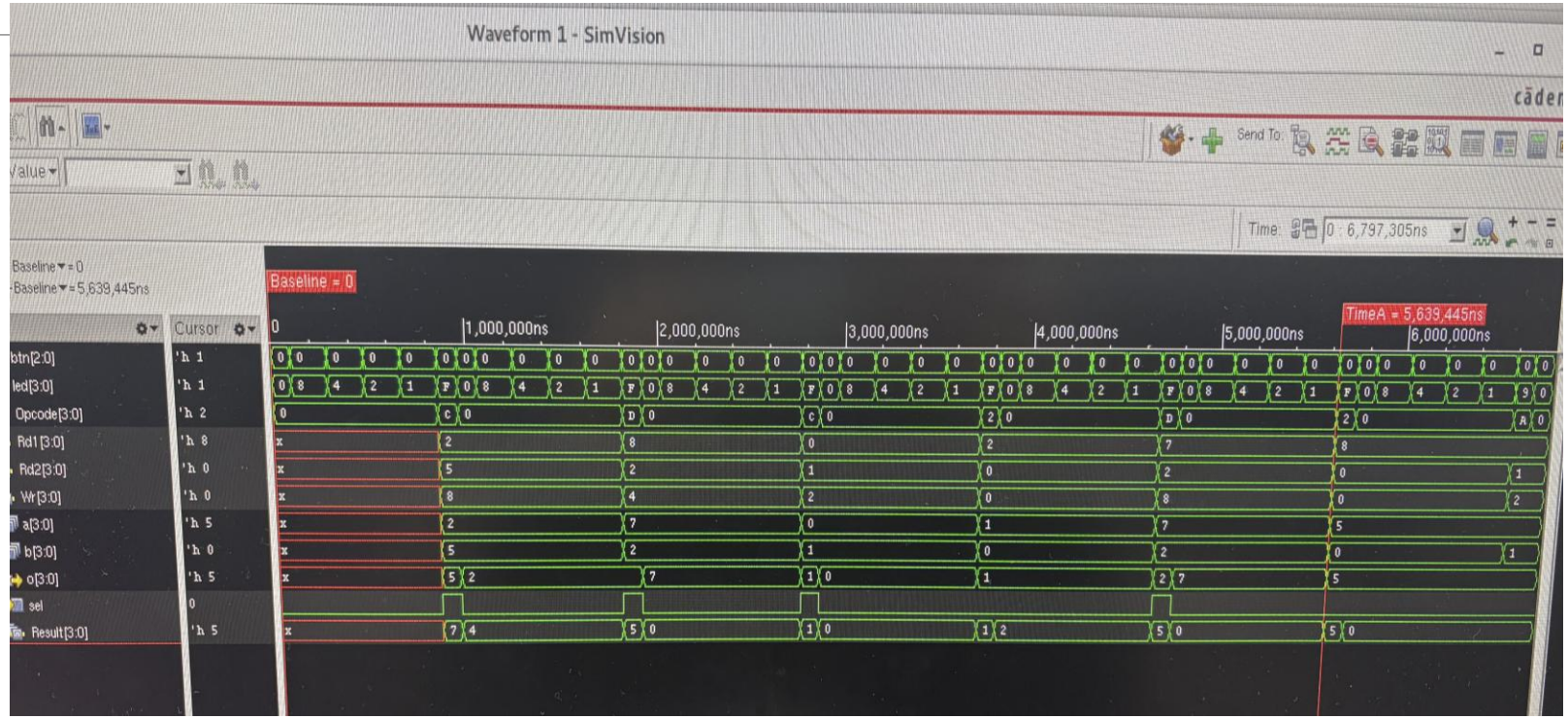
```
77  //todo
78  MUX mux0(.a(Data2),.b(Rd2),.sel(ALU_Src),.o(out));
79
80  //assign out =(ALU_Src == 1)?
81  //
```



사용자의 입력을 그대로 받아
데이터로 사용

Final Project Milestone3





ADDI, SUBI 구현영상과 시뮬레이션.

Opcode는 addi(c), Rd1은 2번 주소지의 데이터(2), Rd2는 사용자의 입력값(5), Wr은 8번 주소지에 저장

Final Project Milestone4

RTL 코드

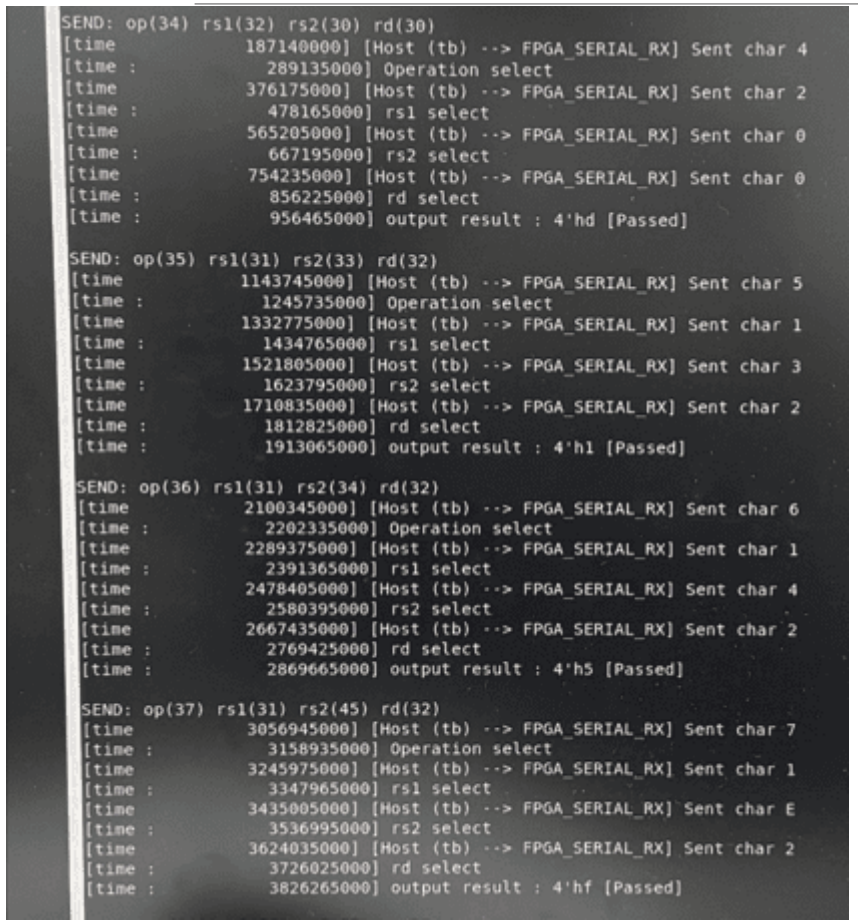
ALU에 들어온 인풋 데이터들 (ALU1, ALU2)의 연산을 통해 tmp에 저장

Add와 sub를 하나의 가산기로 구현하기 위해 m 사용. M=1 : 2의 보수

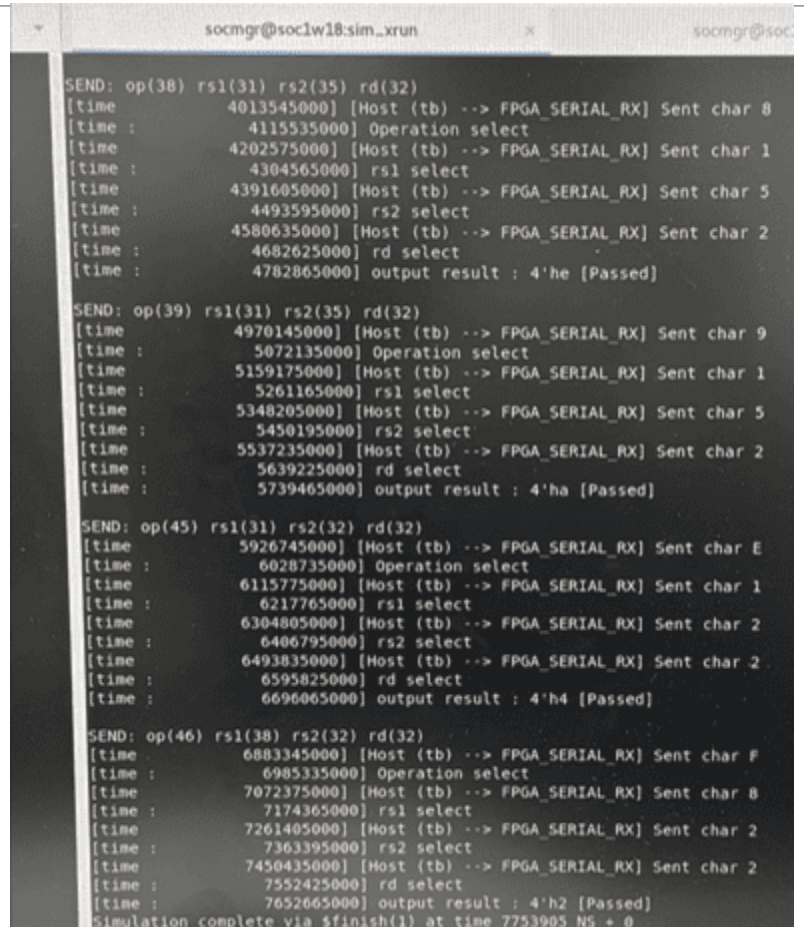
(add_sub코드에서 확인가능)

```
e  Edit  View  Search  Terminal  Help
1  `timescale 1ns / 1ps
2
3  module ALU(                          // create ALU module
4      input [3:0] ALU_Op,              // declare input signal 4bit array ALU_Op for operation
5      input [3:0] ALU1,                // declare input signal 4bit array ALU1 for operand1
6      input [3:0] ALU2,                // declare input signal 4bit array ALU2 for operand2
7
8      output [3:0] ALU_Result,          // declare output signal 4bit array ALU_Result for result
9      output       ov_result
10     );
11
12     reg m;
13     reg [3:0] tmp;                   // declare register type internal signal 4bit array tmp for temporary storage
14     wire [3:0] sum;                  // declare wire type internal signal 4bit array sum for sum
15     wire ov;                         // declare wire type internal signal ov for overflow
16
17     always @(*)                      // Always execute when ALU_Op is changed
18     begin
19         case(ALU_Op)                     // Case ALU_Op
20             1: tmp = ALU2;               // WRITE: Store ALU2 into tmp
21             2: tmp = ALU1;               // READ: Store ALU1 into tmp
22             3: tmp = ALU1;               // COPY: Store ALU1 into tmp
23             4: tmp = ~ALU1;// NOT: Bitwise NOT of ALU1 stored in tmp
24             5: tmp = ALU1 & ALU2;// AND: Bitwise AND of ALU1 and ALU2 stored in tmp
25             6: tmp = ALU1 | ALU2;// OR: Bitwise OR of ALU1 and ALU2 stored in tmp
26             7: tmp = ALU1 ^ ALU2;// XOR: Bitwise XOR of ALU1 and ALU2 stored in tmp
27             8: tmp = ~(ALU1 & ALU2);// NAND: Bitwise NAND of ALU1 and ALU2 stored in tmp
28             9: tmp = ~(ALU1 | ALU2);// NOR: Bitwise NOR of ALU1 and ALU2 stored in tmp
29             10: m = 0;   // ADD: Store the sum in tmp
30             11: m = 1;   // SUB: Store the difference in tmp
31             12: m = 0;   // ADD IMMEDIATE: Store the sum in tmp
32             13: m = 1;   // SUB IMMEDIATE: Store the difference in tmp
33             14: tmp = ALU1 << ALU2;// LEFT SHIFT: Store the left shift result in tmp
34             15: tmp = ALU1 >> ALU2;// RIGHT SHIFT: Store the right shift result in tmp
35         endcase
36     end
37
38     rc_add_sub u0(.a(ALU1), .b(ALU2), .m(m), .sum(sum), .overflow(ov), .cout() );
39
40     assign ALU_Result = ((ALU_Op >= 4'd1 && ALU_Op<= 4'd9) || (ALU_Op >= 4'd14 && ALU_Op<= 4'd15)) ? tmp : (ov) ? 4'b0000 : sum;
41
42     assign ov_result = ov && ((ALU_Op == 4'd10 || ALU_Op == 4'd11 || ALU_Op == 4'd12 || ALU_Op == 4'd13));
43 endmodule
44
```

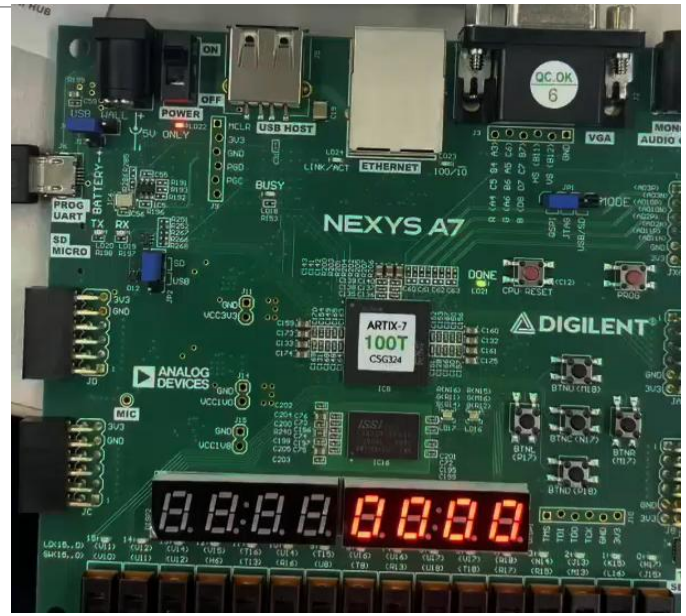
Final Project Milestone4



```
SEND: op(34) rs1(32) rs2(30) rd(30)
[time            187140000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 4
[time :            289135000] Operation select
[time            376175000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :            478165000] rs1 select
[time            565205000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 0
[time :            667195000] rs2 select
[time            754235000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 0
[time :            856225000] rd select
[time :            956465000] output result : 4'hd [Passed]

SEND: op(35) rs1(31) rs2(33) rd(32)
[time           1143745000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 5
[time :           1245735000] Operation select
[time           1332775000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           1434765000] rs1 select
[time           1521805000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 3
[time :           1623795000] rs2 select
[time           1710835000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           1812825000] rd select
[time :           1913065000] output result : 4'h1 [Passed]

SEND: op(36) rs1(31) rs2(34) rd(32)
[time           2100345000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 6
[time :           2202335000] Operation select
[time           2289375000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           2391365000] rs1 select
[time           2478405000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 4
[time :           2580395000] rs2 select
[time           2667435000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           2769425000] rd select
[time :           2869665000] output result : 4'h5 [Passed]

SEND: op(37) rs1(31) rs2(45) rd(32)
[time           3056945000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 7
[time :           3158935000] Operation select
[time           3245975000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           3347965000] rs1 select
[time           3435005000] [Host (tb) --> FPGA_SERIAL_RX] Sent char E
[time :           3536995000] rs2 select
[time           3624035000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           3726025000] rd select
[time :           3826265000] output result : 4'hf [Passed]
```



```
socmgr@soc1w18:sim_xrun

SEND: op(38) rs1(31) rs2(35) rd(32)
[time           4013545000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 8
[time :           4115535000] Operation select
[time           4202575000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           4304565000] rs1 select
[time           4391605000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 5
[time :           4493595000] rs2 select
[time           4580635000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           4682625000] rd select
[time :           4782865000] output result : 4'he [Passed]

SEND: op(39) rs1(31) rs2(35) rd(32)
[time           4970145000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 9
[time :           5072135000] Operation select
[time           5159175000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           5261165000] rs1 select
[time           5348205000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 5
[time :           5450195000] rs2 select
[time           5537235000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           5639225000] rd select
[time :           5739465000] output result : 4'ha [Passed]

SEND: op(45) rs1(31) rs2(32) rd(32)
[time           5926745000] [Host (tb) --> FPGA_SERIAL_RX] Sent char E
[time :           6028735000] Operation select
[time           6115775000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           6217765000] rs1 select
[time           6304805000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           6406795000] rs2 select
[time           6493835000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           6595825000] rd select
[time :           6696065000] output result : 4'h4 [Passed]

SEND: op(46) rs1(38) rs2(32) rd(32)
[time           6883345000] [Host (tb) --> FPGA_SERIAL_RX] Sent char F
[time :           6985335000] Operation select
[time           7072375000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 8
[time :           7174365000] rs1 select
[time           7261405000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           7363395000] rs2 select
[time           7450435000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           7552425000] rd select
[time :           7652665000] output result : 4'h2 [Passed]
Simulation complete via $finish(1) at time 7753905 NS + 0
```



Opcode : Left shift
Rd1 : 1번 주소지 데이터
Rd2 : 사용자 입력값
Wr : 2번 주소지에 저장
0001 ----- > 0100

Final Project Milestone5

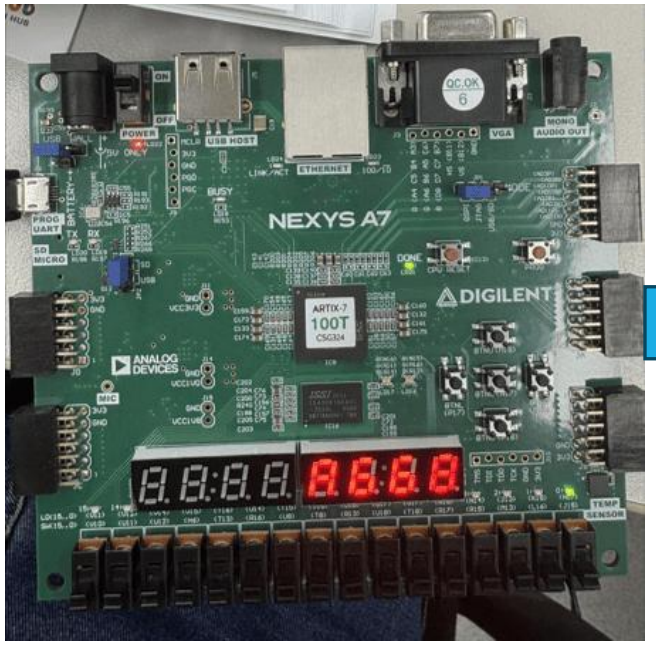
```
`timescale 1ns / 1ps

module hex2ssd(                 // create HEX_to_SSD module
    input       [4:0] hex,      // declare input signal array 'HEX'
    output reg  [6:0] seg       // declare output signal array type
    );

    always @(*)                 // executes when input signal change
    begin
        case(hex)               // case for input signal 'BCD'
            5'h0 : seg = 7'h40;     // when HEX is 0
            5'h1 : seg = 7'h79;     // when HEX is 1
            5'h2 : seg = 7'h24;     // when HEX is 2
            5'h3 : seg = 7'h30;     // when HEX is 3
            5'h4 : seg = 7'h19;     // when HEX is 4
            5'h5 : seg = 7'h12;     // when HEX is 5
            5'h6 : seg = 7'h02;     // when HEX is 6
            5'h7 : seg = 7'h78;     // when HEX is 7
            5'h8 : seg = 7'h00;     // when HEX is 8
            5'h9 : seg = 7'h10;     // when HEX is 9
            5'ha : seg = 7'h08;     // when HEX is a
            5'hb : seg = 7'h03;     // when HEX is b
            5'hc : seg = 7'h46;     // when HEX is c
            5'hd : seg = 7'h21;     // when HEX is d
            5'he : seg = 7'h06;     // when HEX is e
            5'hf : seg = 7'h0e;     // when HEX is f
            5'd16 : seg = 7'h41;  //v
            5'd17 : seg = 7'h0f;  //r
        endcase
    end

endmodule
```

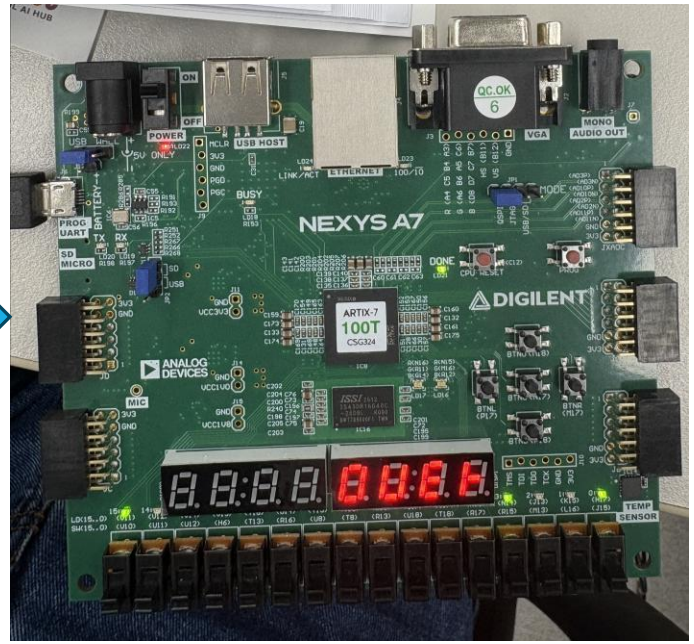
```
end

assign res0 = (Result[0] == 1'b1) ? 4'b0001 : 4'b0000;        // if Result[0] is 1'b1, res0 is 000
assign res1 = (Result[1] == 1'b1) ? 4'b0001 : 4'b0000;        // if Result[1] is 1'b1, res1 is 000
assign res2 = (Result[2] == 1'b1) ? 4'b0001 : 4'b0000;        // if Result[2] is 1'b1, res2 is 000
assign res3 = (Result[3] == 1'b1) ? 4'b0001 : 4'b0000;        // if Result[3] is 1'b1, res3 is 000

assign HEXA = (state != 5 && state != 6) ? temp1 : (btn[1]) ? temp1 :(state==6)?5'h0 : res3; // if
assign HEXB = (state != 5 && state != 6) ? temp2 : (btn[1]) ? temp2 :(state==6)?5'd16: res2; // if
assign HEXC = (state != 5 && state != 6) ? temp3 : (btn[1]) ? temp3 :(state==6)?5'he : res1; // if
assign HEXD = (state != 5 && state != 6) ? temp4 : (btn[1]) ? temp4 :(state==6)?5'd17: res0; // if

hex2ssd h0(.hex(HEXA), .seg(SSDA));                    // HEX_to_SSD(h0) makes SSDA(7 bit) sign
hex2ssd h1(.hex(HEXB), .seg(SSDB));                    // HEX_to_SSD(h1) makes SSDB(7 bit) sign
hex2ssd h2(.hex(HEXC), .seg(SSDC));                    // HEX_to_SSD(h2) makes SSDC(7 bit) sign
hex2ssd h3(.hex(HEXD), .seg(SSDD));                    // HEX_to_SSD(h3) makes SSDD(7 bit) sign

reg [1:0] count;
```

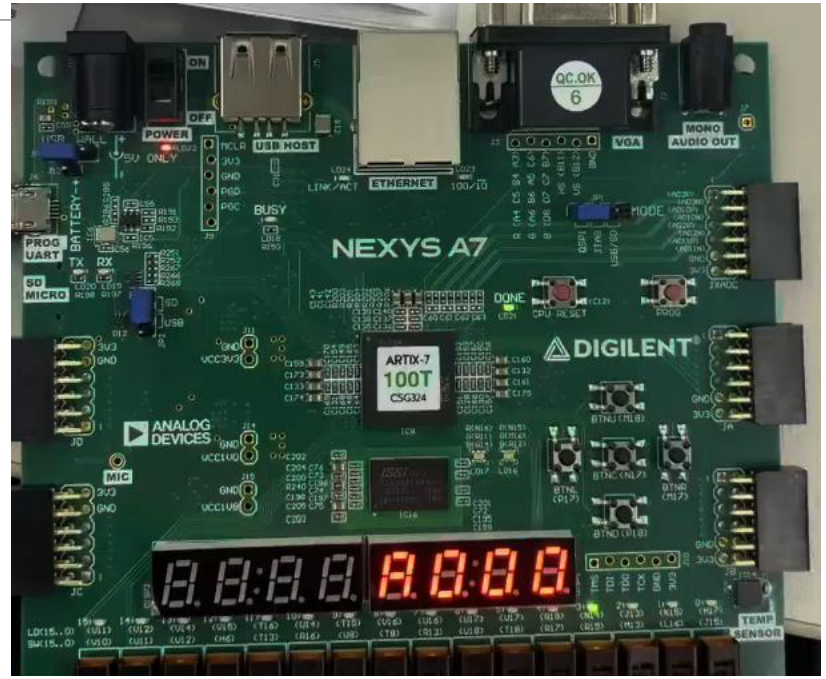
State==6, 즉 오버플로우 발생 상황에서 이전에 매핑해둔 over 출력되도록 코드 변경

글자 구현 : over를 표현할 수 있도록 7-segmen hex를 5비트로 늘려 v, r 매핑

Final Project Milestone5

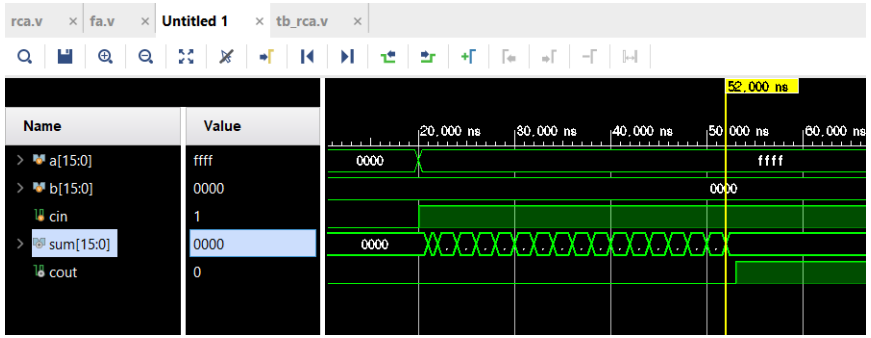


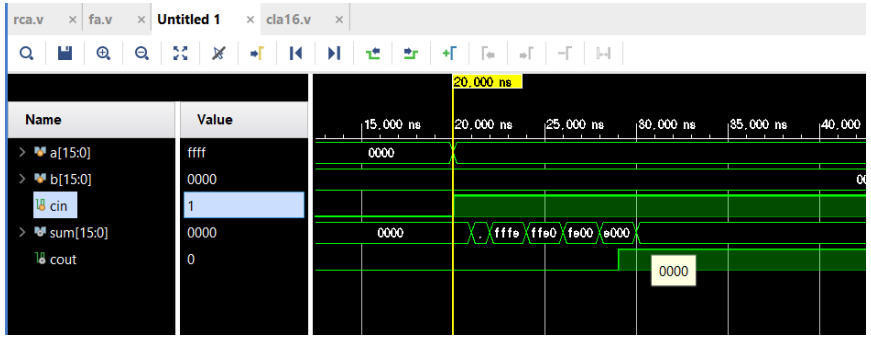




2의 보수 기반 4비트 signed 연산에서 6+6의 결과는 1100으로 나타나며, 최대 양수 0111을 넘어서 부호가 변하므로 오버플로우 발생

RCA – CLA 단일모듈 비교



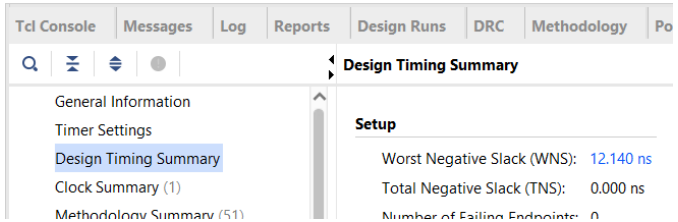


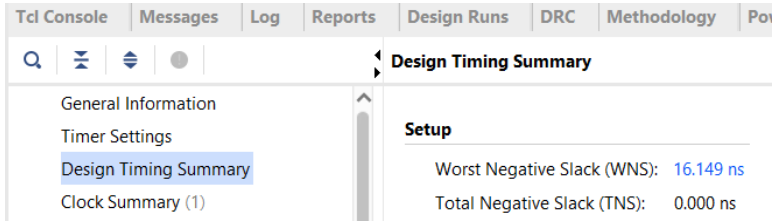
Cout 기준 76% 단축

Resource	Estimation	Available	Utilization...
LUT	16	41000	0.04
IO	50	300	16.67

Resource	Estimation	Available	Utilization...
LUT	29	41000	0.07
FF	50	82000	0.06
IO	52	300	17.33
BUFG	1	32	3.13

LUT 0.03%p 증가





타이밍 마진 33% 확보

개인/프로젝트

PROJECT 01. 공정 미세화 한계 극복을 위한 AXI 기반 SoC 통합 및 PIM 개념의 Near-Memory 하드웨어 가속기 설계

수행기간

2025.12~2026.03

수행인원

3명 (스펙설정 및 모델링, RTL코딩 인터페이스 통합 및 FPGA 디버깅, Testbench 검증 및 보드 테스트)

사용기술

VIVADO(verilog), FPGA, ILA

수행내용

(1) Bottom to top으로 16bit CPU설계 및 UART로 들어오는 입력을 처리하는 제어 로직 구현

(2) 데이터값 및 연산모드 입력받기, 합산, 평균, 중앙값 연산하여 시스템 구축

(3) 커스텀한 CPU와 Xilinx사의 Bram Memory Controller IP와 연결. 버스 프로토콜로는 AXI4-Lite 사용 – 직접 만든 Master와 표준 Slave간의 연결 확인

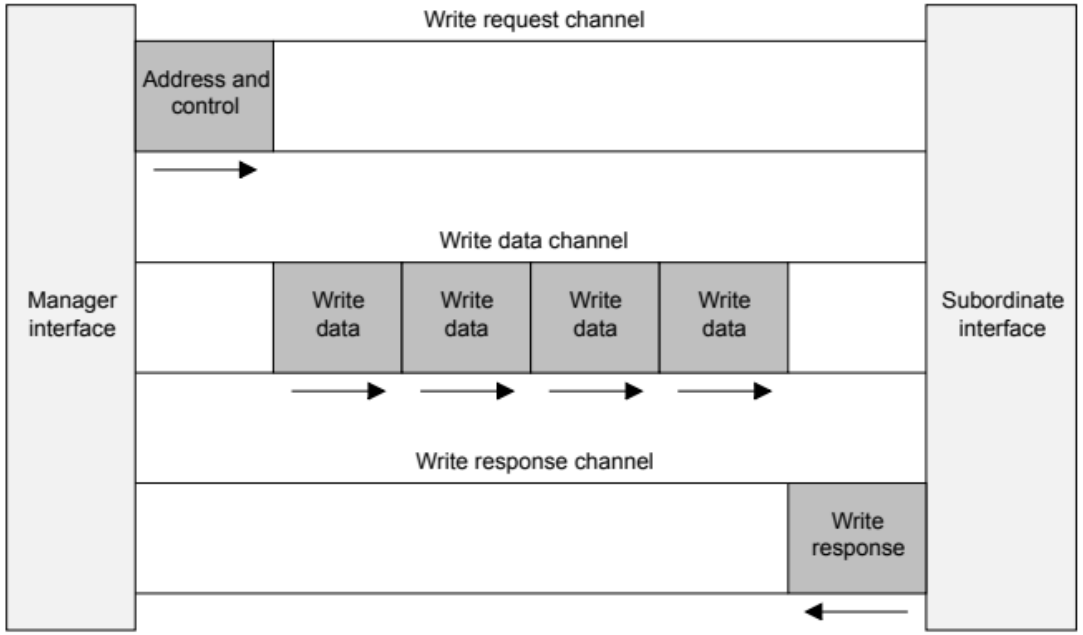
(4) PIM Slave 설계 – 단순 연산 시 CPU의 메모리 병목 해결, MAC 연산으로 AI 가속기로의 확장성 내제

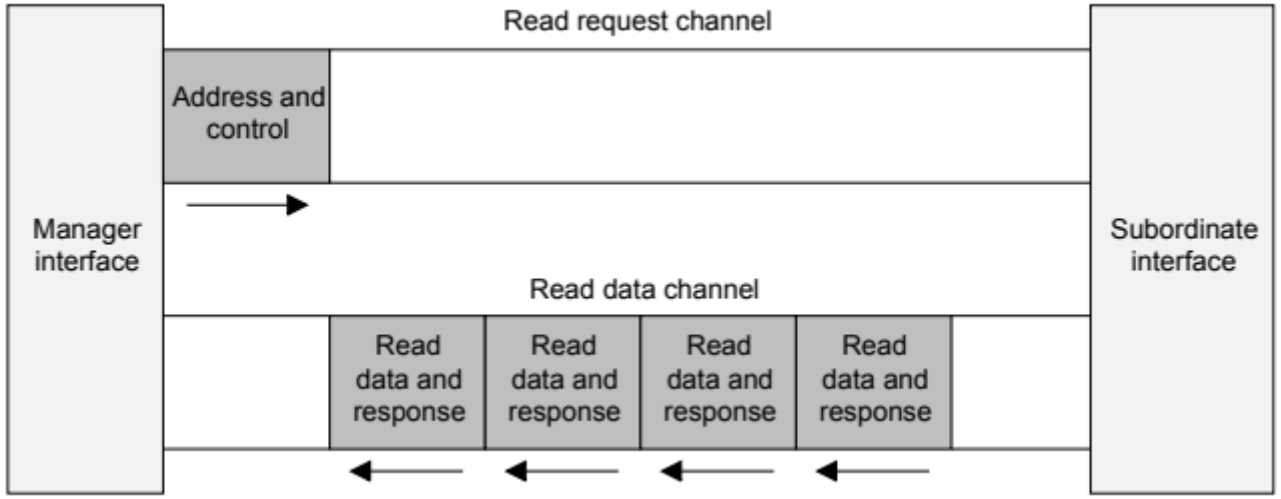
1. 서론 및 스펙 설정

미세 공정으로 갈수록 메모리와 연산기 사이의 물리적 거리에 따른 RC Delay와 Dynamic Power 소모가 극심해짐. 따라서 데이터 이동 자체를 최소화하는 AXI 기반 SoC 환경 구축 및, 메모리 병목을 제거하는 PIM 개념의 하드웨어 가속기 설계를 시도함

분류	스펙	비고
Core Architecture	16-bit Multi-Cycle CPU	커스텀 설계
Instruction Set	16-bit ISA	산술/논리/메모리/제어 명령어 자체 정의
System Bus	AMBA AXI4-Lite	32-bit Address / Data 패스
Memory System	Xilinx Standard BRAM Controller	표준 메모리 접근용 (MMIO 방식)
PIM controller	Custom RTL FSM Controller	커스텀 설계
Accelerator	Near-Memory MAC 가속기	PIM 개념 도입, DSP 할당
I/O Interface	UART (Rx/Tx), 7-Segment, Switch, LED	PC-FPGA 간 직렬 통신 및 상태 모니터링
Operating Freq	50MHz	FPGA보드 기본 클럭 주파수

2. 관련 연구 – AXI



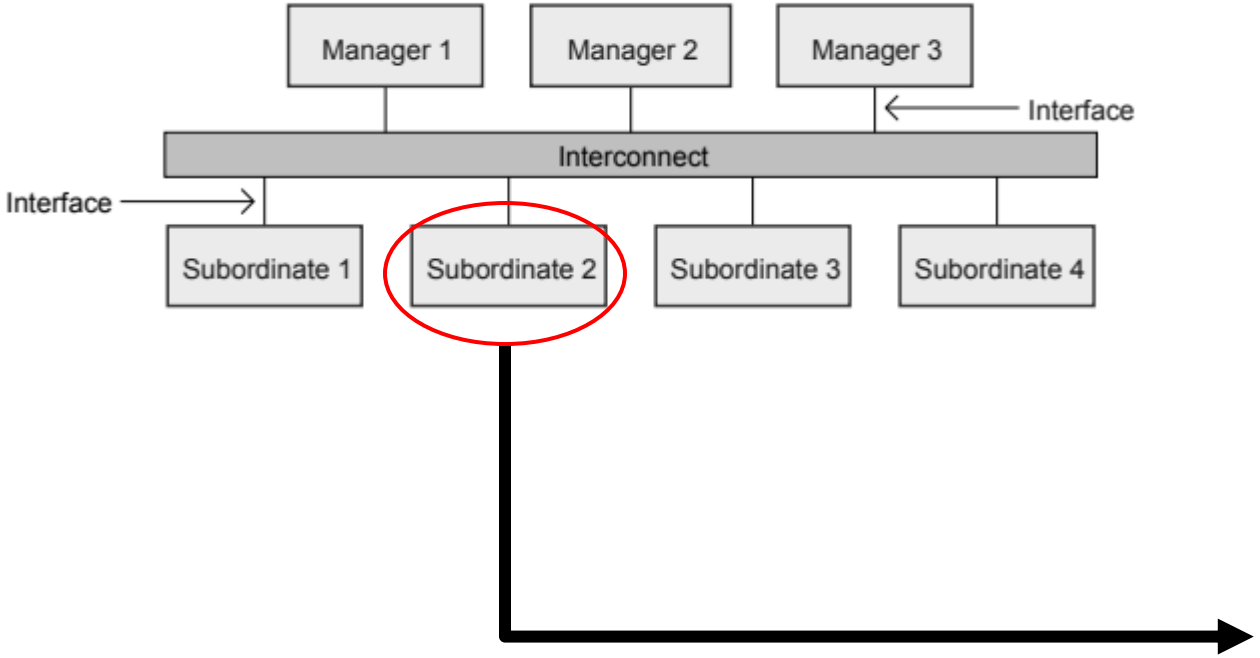


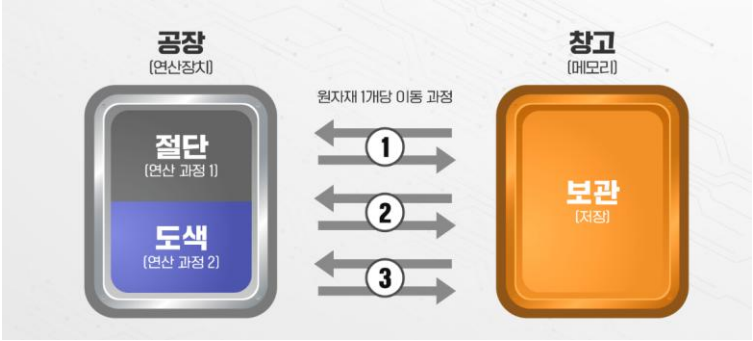
WRITE – AHB와 달리 주소와 함께 미리 데이터 받을 준비 가능

READ – 데이터 값의 도착 여부와 관계없이 주소유지 필수x

출처 : ARM – AMBA AXI Protocal Specification

2. 관련 연구



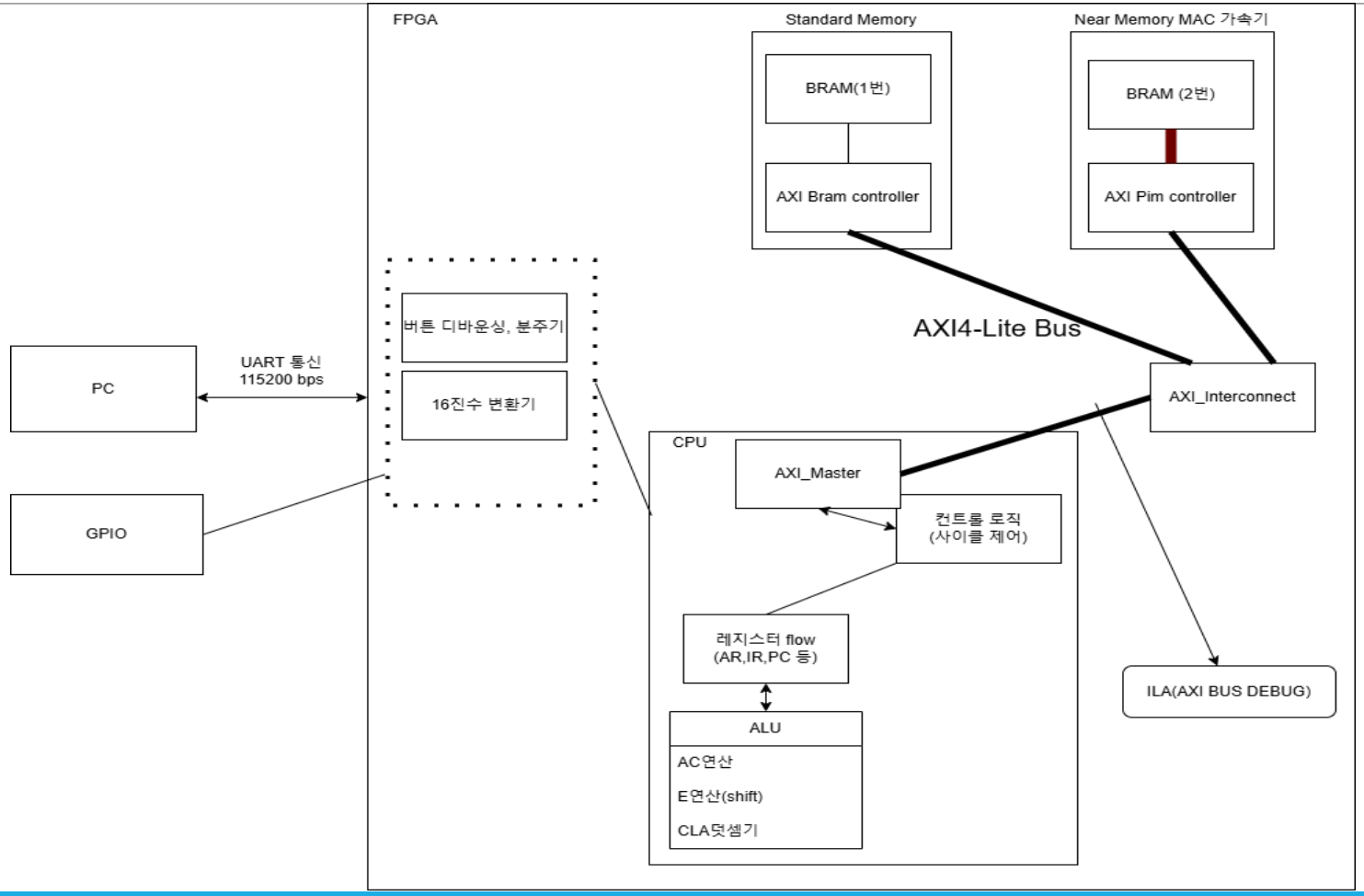


기존 폰노이만 구조

PIM 구조

출처 : SK HYNIX NEWSROOM

3. 블록 다이어그램


FPGA
Standard Memory
BRAM(1번)
AXI Bram controller
Near Memory MAC 가속기
BRAM (2번)
AXI Pim controller
AXI4-Lite Bus
AXI_Interconnect
버튼 디바운싱, 분주기
16진수 변환기
PC
UART 통신
115200 bps
GPIO
CPU
AXI_Master
컨트롤 로직
(사이클 제어)
레지스터 flow
(AR,IR,PC 등)
ALU
AC연산
E연산(shift)
CLA덧셈기
ILA(AXI BUS DEBUG)

4. 기능 정리

1번 모드
- 어셈블리 합산

2번 모드
- 어셈블리 평균

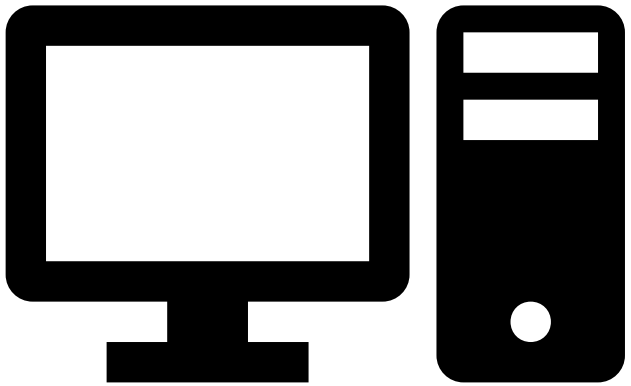
3번 모드
- 어셈블리 중앙값

4번 모드
- Pim 합산

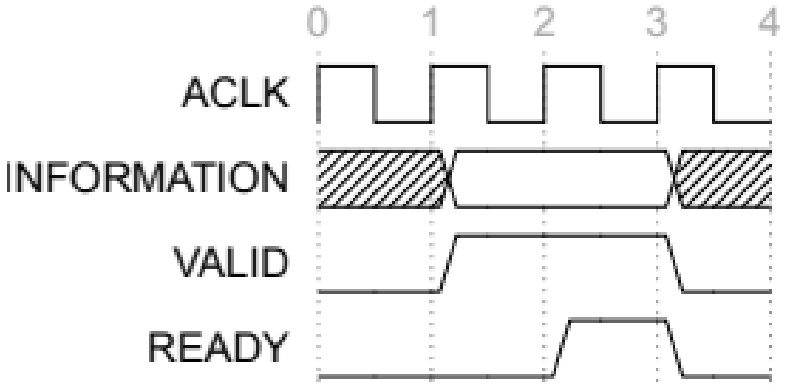
5번 모드
- Pim 평균

6번 모드
- Pim MAC 연산

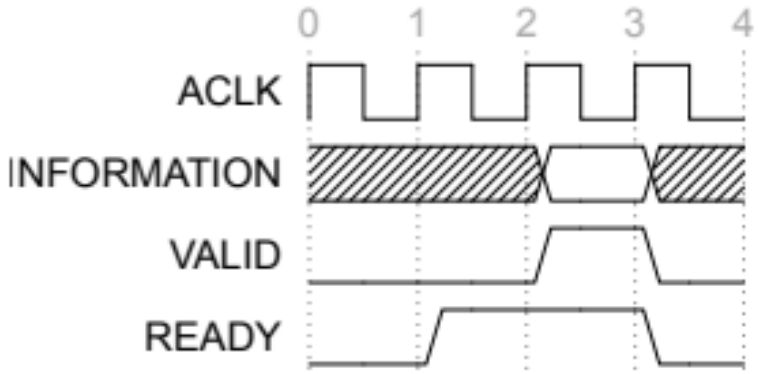
PC로부터 UART 통신으로 모드 및 데이터 입력



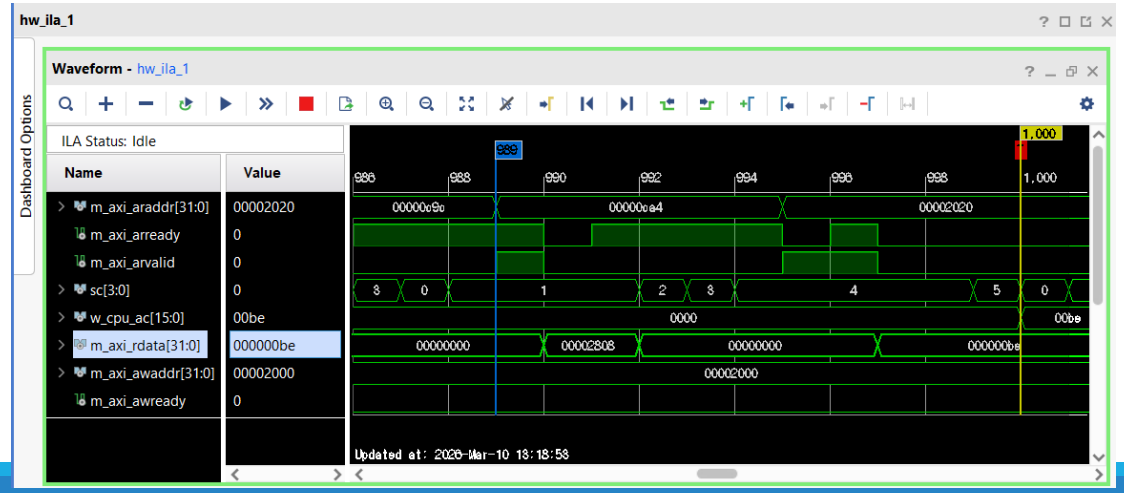
4. 기능 정리

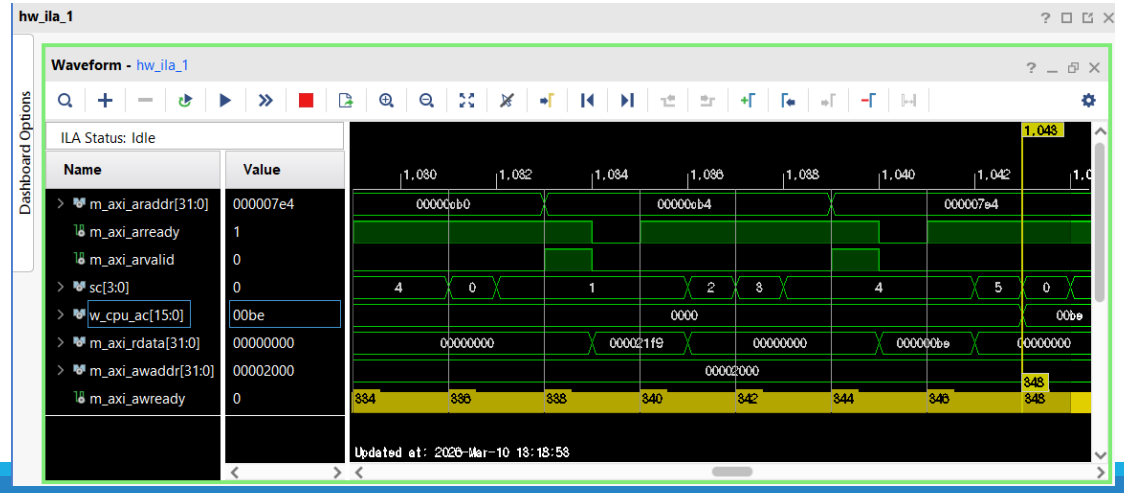


마스터 기준 CPU가 Write할 주소를 넘길 때



마스터 기준 CPU가 Read할 주소를 넘길 때

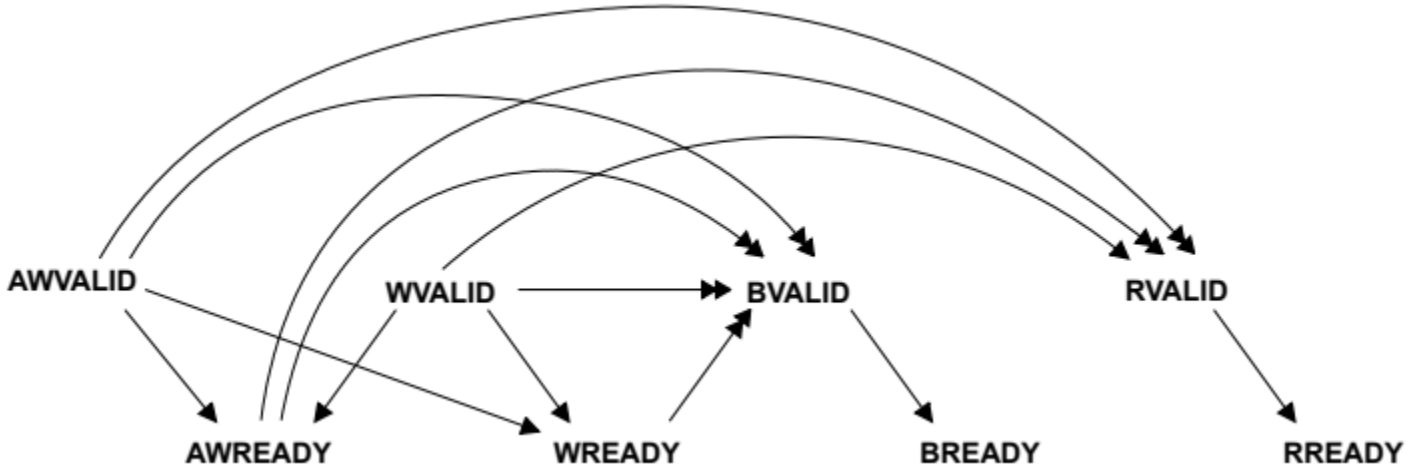




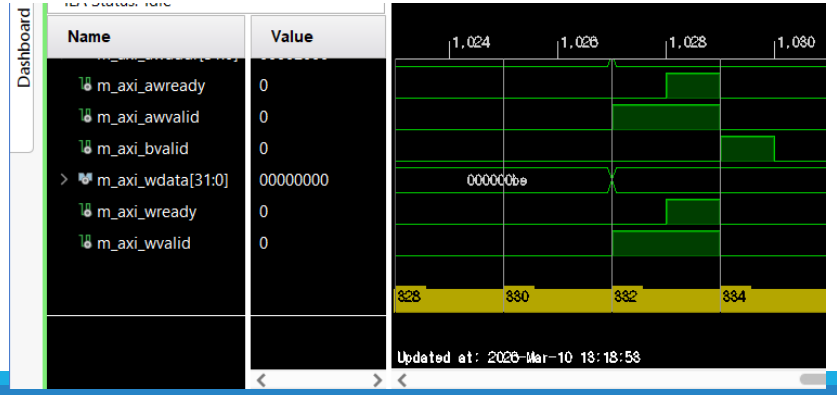
5. 핵심코드

: 서로 얽히지 않는 자유로운 관계

: 먼저 완료되어야 함 (데드락 방지)




```
53          if (s_axi_awvalid && s_axi_wvalid && !s_axi_awready && !s_axi_wready) begin
54              s_axi_awready <= 1'b1;
55              s_axi_wready  <= 1'b1;
56              write_accept  <= 1'b1;
57
58              case (s_axi_awaddr[7:0])
59                  8'h00: slv_reg0_ctrl <= s_axi_wdata;
60              endcase
61          end
62          else begin
63              s_axi_awready <= 1'b0;
64              s_axi_wready  <= 1'b0;
65              write_accept <= 1'b0;
66          end
67
68          if (write_accept && !s_axi_bvalid) begin
69              s_axi_bvalid <= 1'b1;
70          end
71          else if (s_axi_bready && s_axi_bvalid) begin
72              s_axi_bvalid <= 1'b0;
73          end
```



BVALID=1 조건 : AWVALID, WVALID, AWREADY, WREADY 모두 1인 상태

5. 핵심코드

Project Summary × pim_slave.v × cpu.v × axi_master.v ×

C:/Users/user/cpu_project/cpu_project.srcs/sources_1/new/axi_master.v

```
44          case(state)
45              IDLE: begin
46                  if(cpu_req) begin
47                      if(cpu_we) begin
48                          state <= WRITE_ADDR;
49                          m_axi_awaddr <= {18'b0, cpu_addr, 2'b00}; m_axi_awvalid <= 1;
50                          m_axi_wdata  <= {16'b0, cpu_wdata}; m_axi_wvalid <= 1;
51                          m_axi_wstrb  <= 4'b0011;
52                          m_axi_bready <= 1;
53                      end
54                      else begin
55                          state <= READ_ADDR;
56                          m_axi_araddr <= {18'b0, cpu_addr, 2'b00};
57                          m_axi_arvalid <= 1;
58                          m_axi_rready <= 1;
59                      end
```

Project Summary × Device × interconnect.v × axi_master.v × cpu.v ×

C:/Users/user/cpu_desing/cpu_desing.srcs/sources_1/new/cpu.v

```
210         .doutb()
211     );
212
213
214
215     blk_mem_gen_0 u_ram(
216         .clka(bram_clk_a),
217         .wea(bram_we_a[0]),
218
219         .addra(bram_addr_a[14:2]),
220
221         .dina(bram_wrdata_a),
222         .douta(bram_rddata_a),
223
```

버스로 이동 시 주소*4 ← AXI는 바이트 단위의 읽기 → 버스에서 메모리로 이동 시 주소/4

5. 핵심코드

Project Summary × pim_slave.v × cpu.v × axi_master.v × **interconnect.v**

C:/Users/user/cpu_project/cpu_project.srcs/sources_1/new/interconnect.v

```
67
68      input  wire [31:0] m01_axi_rdata,
69      input  wire [1:0]  m01_axi_rresp,
70      input  wire        m01_axi_rvalid,
71      output wire        m01_axi_rready
72  );
73
74      wire is_write_for_pim = (s_axi_awaddr >= 32'h0000_2000);
75      wire is_read_for_pim  = (s_axi_araddr >= 32'h0000_2000);
76
```

(10진수 2048번지부터 PIM으로 이동)

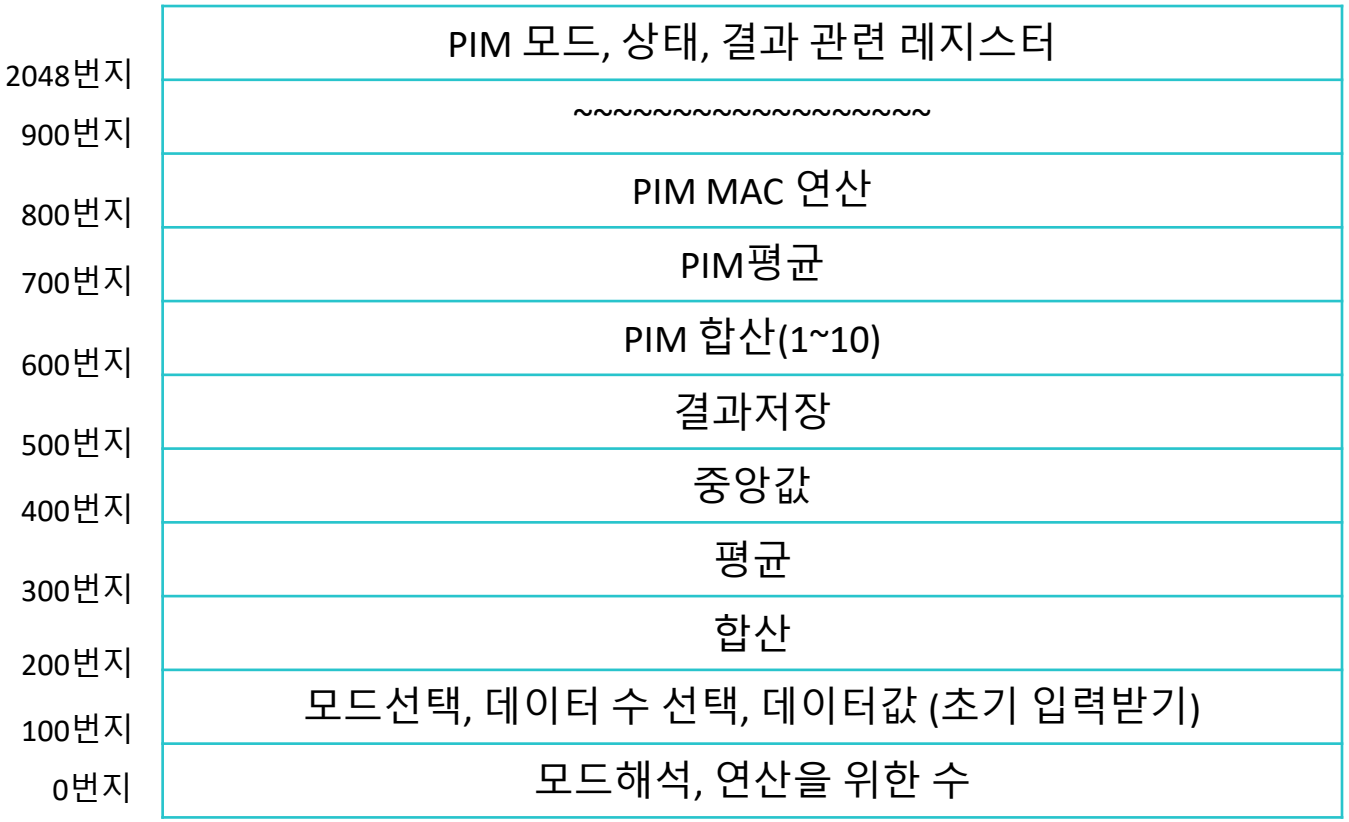
Project Summary × Device × interconnect.v × axi_master.v × cpu.v × **pim_slave.v** ×

C:/Users/user/cpu_project/cpu_project.srcs/sources_1/new/pim_slave.v

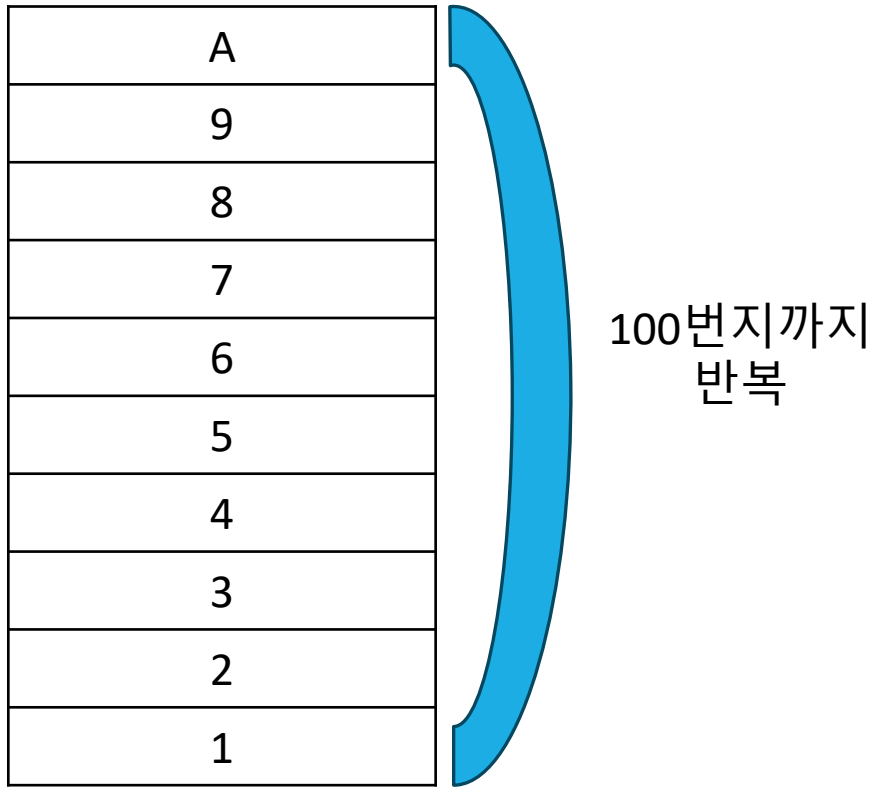
```
157             end
158             READ_2 : begin
159                 state <= CALC2;
160             end
161             CALC2 : begin
162                 mac_res <= mac_res + (mac_reg*bram_rddata);
163                 data_count <= data_count +1;
164                 if (data_count == 32'd9) begin
165                     state <= DONE;
166                 end
167                 else begin
168                     bram_addr <= bram_addr + 32'd4;
169                     state <= READ_MEM;
170                 end
```

(MAC 연산 코드)

6. 메모리맵



메인 메모리



PIM 연동형 온칩 메모리

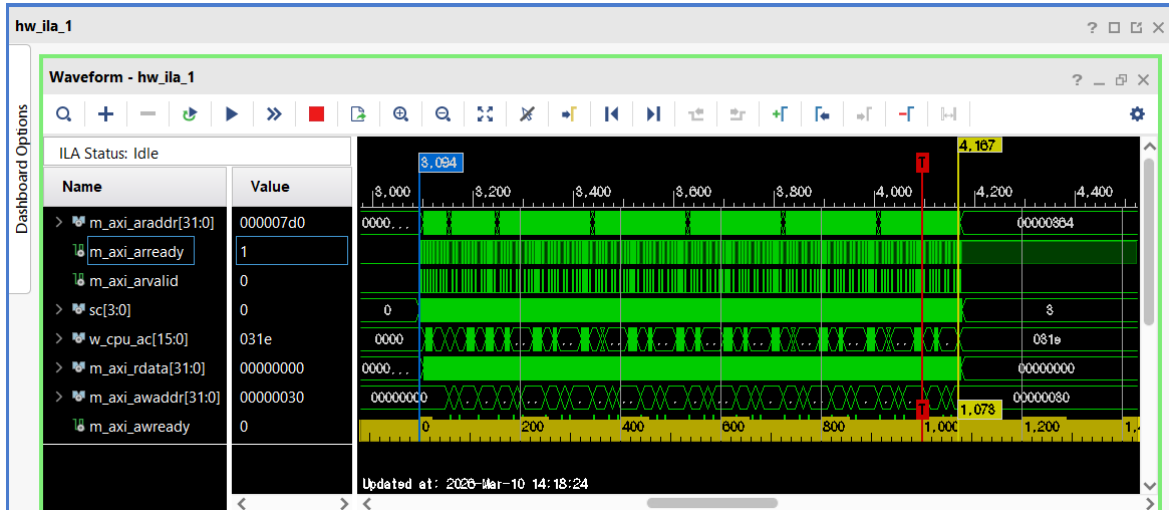
7. 결과(FPGA)

어셈블리 합산 결과

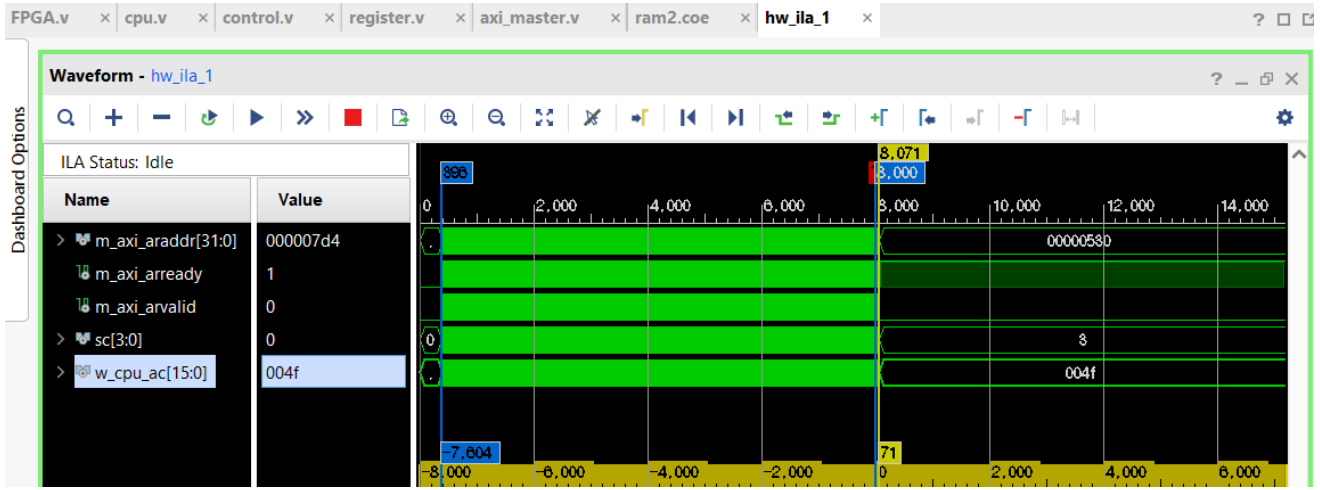
어셈블리 평균 결과

어셈블리 중앙값 결과

7. 결과(ILA 파형)



어셈블리 합산 1073 사이클 소요

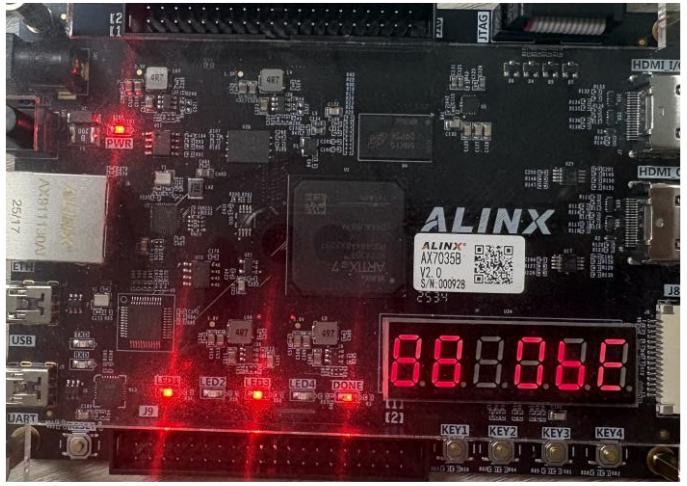


어셈블리 평균 7673 사이클 소요

7. 결과(FPGA)

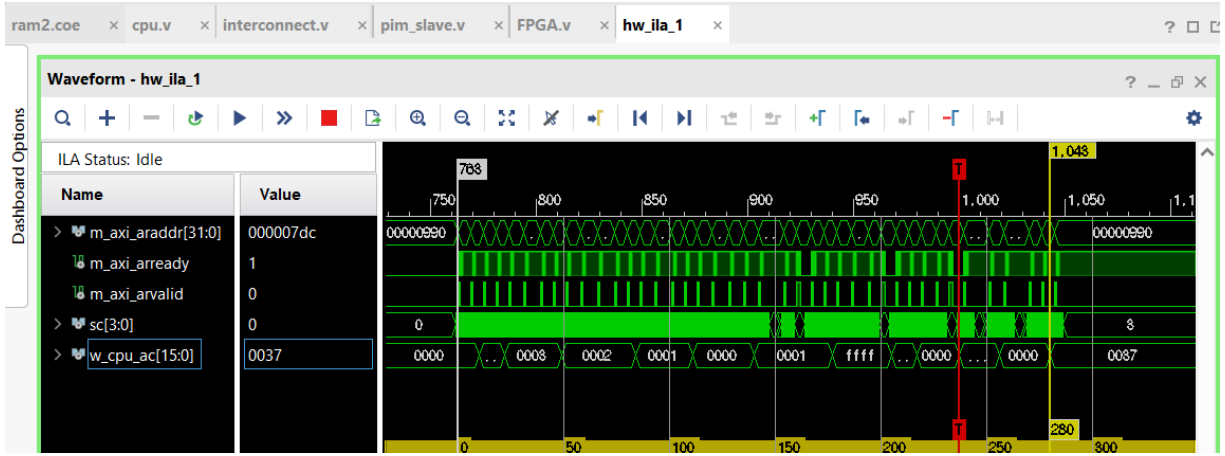
PIM 합산

PIM 평균

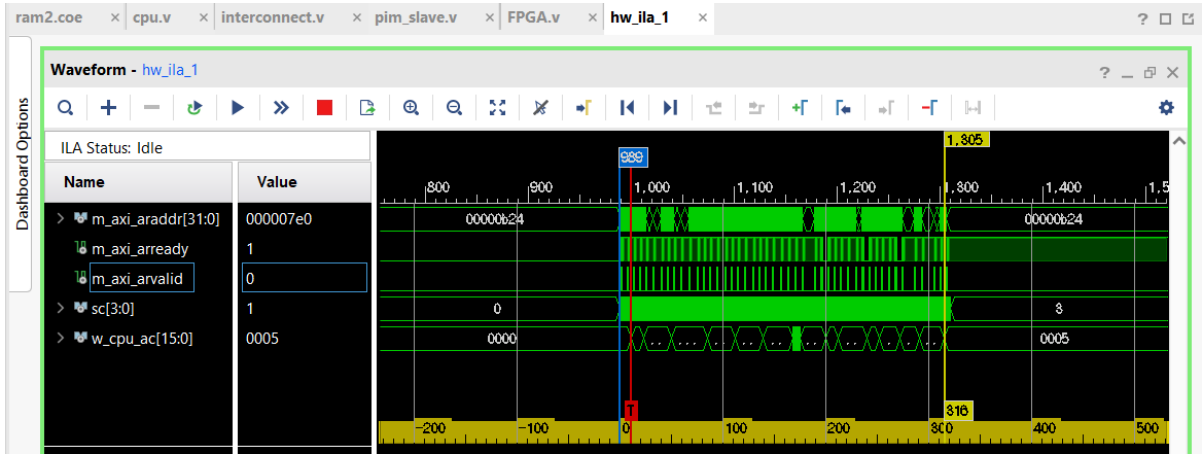


PIM MAC 연산

7. 결과(ILA 파형)

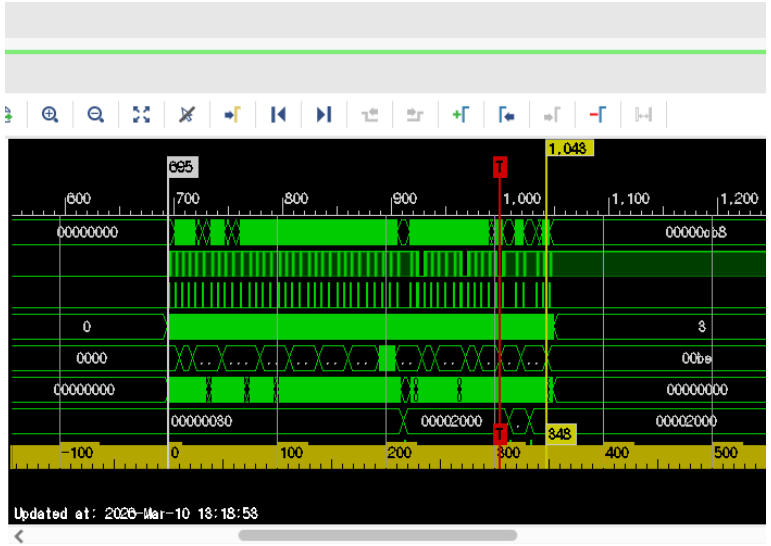


PIM 합산 280 사이클 소요

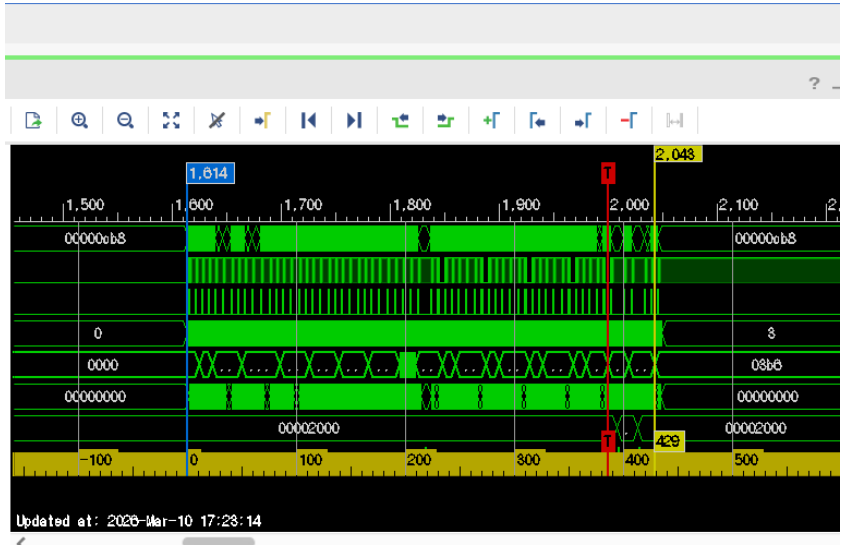


PIM 평균 316 사이클 소요

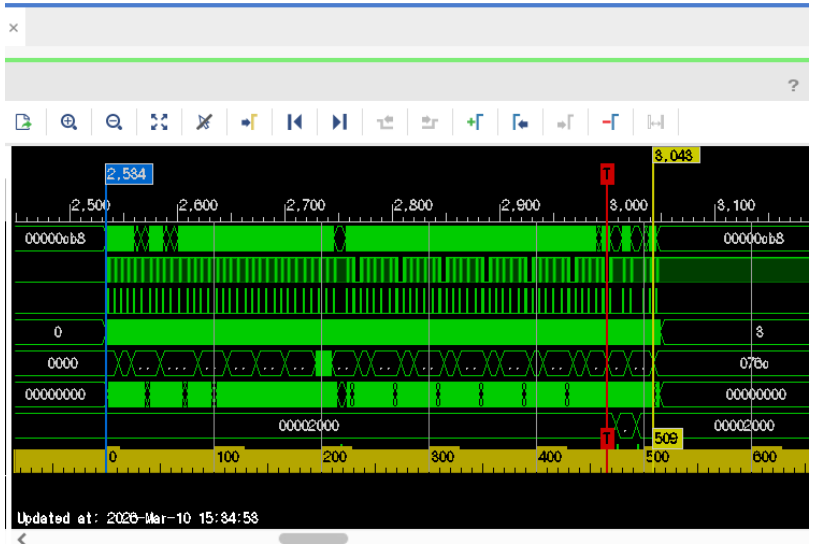
7. 결과(ILA 파형 – MAC연산)



10개 데이터
348사이클 소요

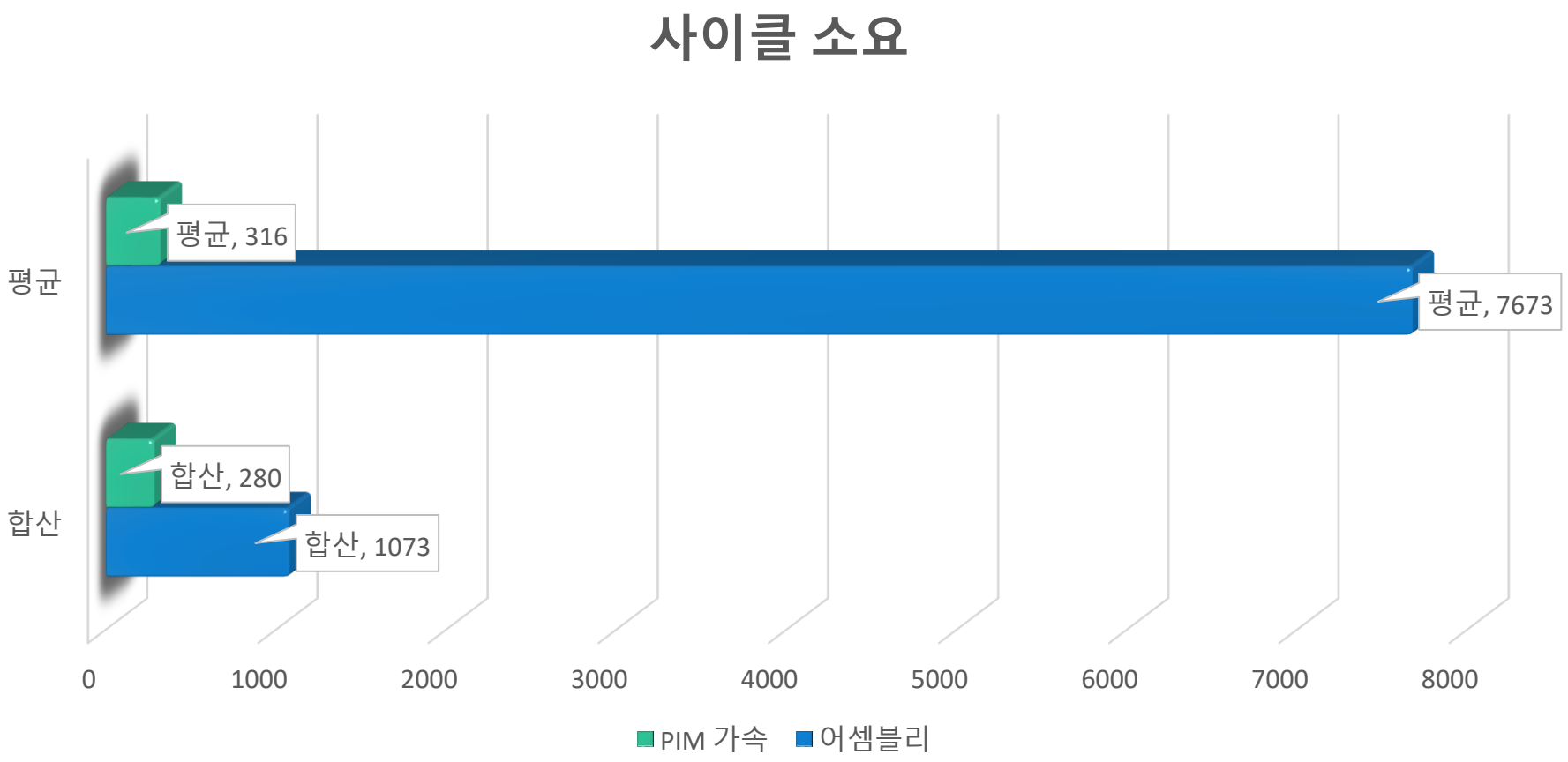


50개 데이터
429사이클 소요

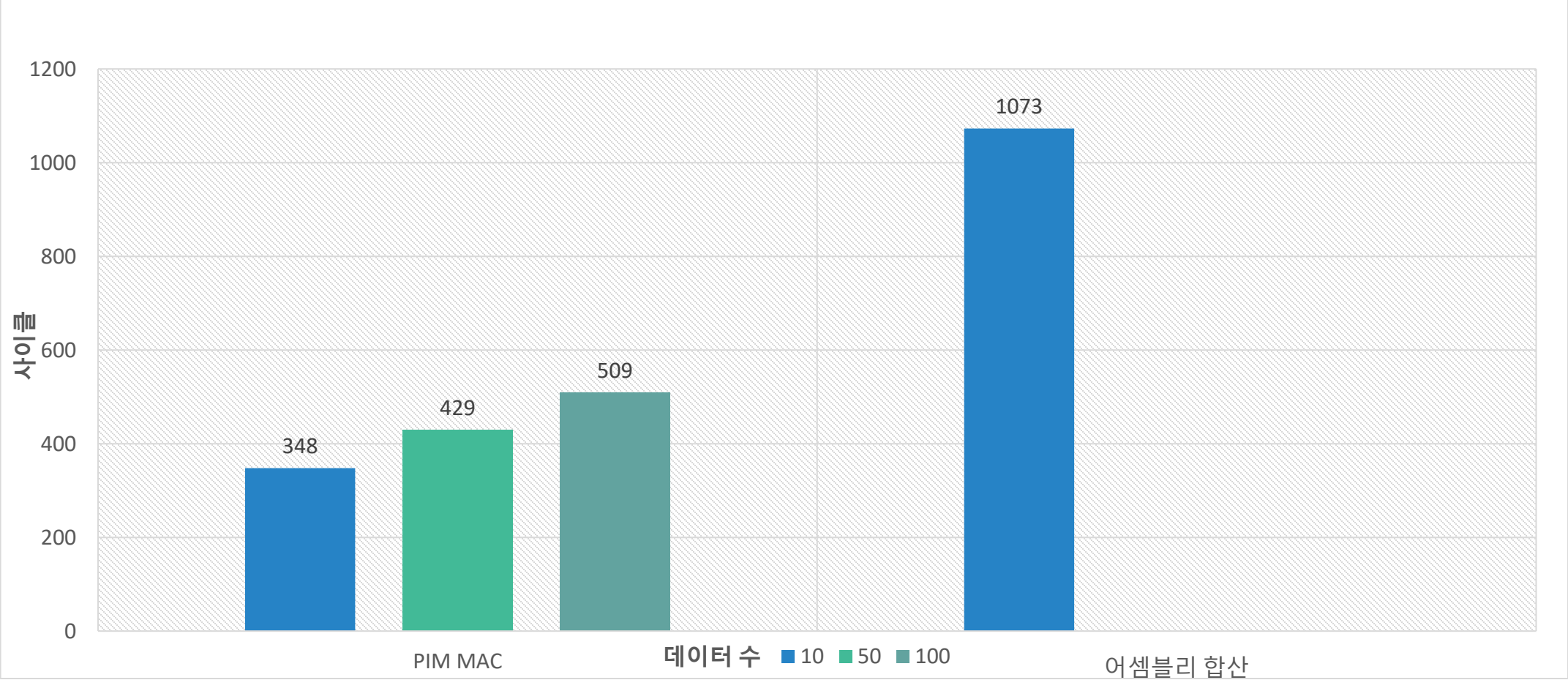


100개 데이터
509사이클 소요

8. 결론 (사이클 비교)


사이클 소요
평균
합산
평균, 316
평균, 7673
합산, 280
합산, 1073
0
1000
2000
3000
4000
5000
6000
7000
8000
PIM 가속
어셈블리

8. 결론 (기존 메모리 연산 대비 MAC 연산 사이클 비교)



8. 결론 (MAC)

MAC연산 :

- 기계학습을 가능케 하는 알고리즘은 복잡하지는 않은 연산의 반복처리가 필요하다. 두 벡터를 내적하고 그 값들을 합하는 MAC은 AI가속에 필수적이다.

- 기존의 폰노이만 CPU는 연산을 수행하기 위해 메인 버스(AXI)를 통해 끊임없이 데이터를 병목 구간으로 끌어와야 하는 한계에 직면해 있다. 따라서 본 프로젝트에서는 시스템 버스의 간섭 없이 메모리 내부에서 직접 AI 핵심 연산을 초고속으로 처리하는 PIM기반의 하드웨어 가속기를 설계하여 그 효용성을 증명한다.

- 이 구조는 상위 메탈 레이어의 사용을 줄이고 BEOL 라우팅 혼잡도와 발열을 분산시키는 공정 친화적 설계의 장점을 가진다.

- 실제 ASIC 양산 시 넓은 Die Size를 차지하는 복잡한 버스 인터커넥트와 대형 캐시 메모리를 배제하여 면적 효율성 및 DFM에 기여할 수 있다.

- 실시간 처리가 생명인 자율주행이나 엣지 AI환경에서 처리 시간을 보장할 수 있다.

8. 결론 (MAC 연산 Throughput)

$$\text{Throuput} = \frac{\text{처리 데이터 수}}{\text{총 } CYCLE}$$

10	• 0.0287
50	• 0.1166
100	• 0.1965

ILA 실측 결과, 입력 데이터 크기가 증가할수록 초기 latency의 영향이 감소하며, 추가 데이터 처리에 필요한 cycle은 약 2 Cycle/Data로 관측되었다.

Throuput의 증가를 확인하며 이는 본 MAC 구조가 데이터 크기가 증가할수록 안정적인 지속 처리율에 수렴함을 보여준다.

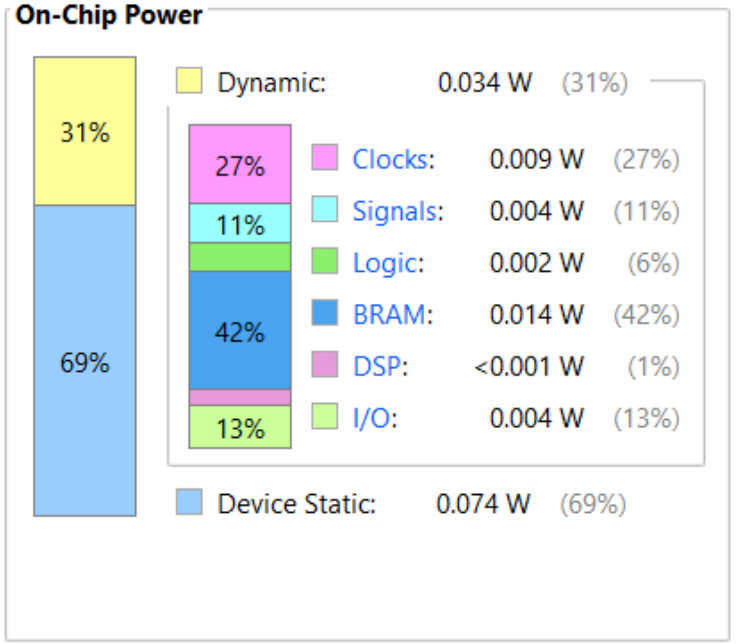
8. 결론(전력소모 및 에너지)

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.108 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.3°C**
Thermal Margin:	59.7°C (21.2 W)
Effective ϑJA:	2.8°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

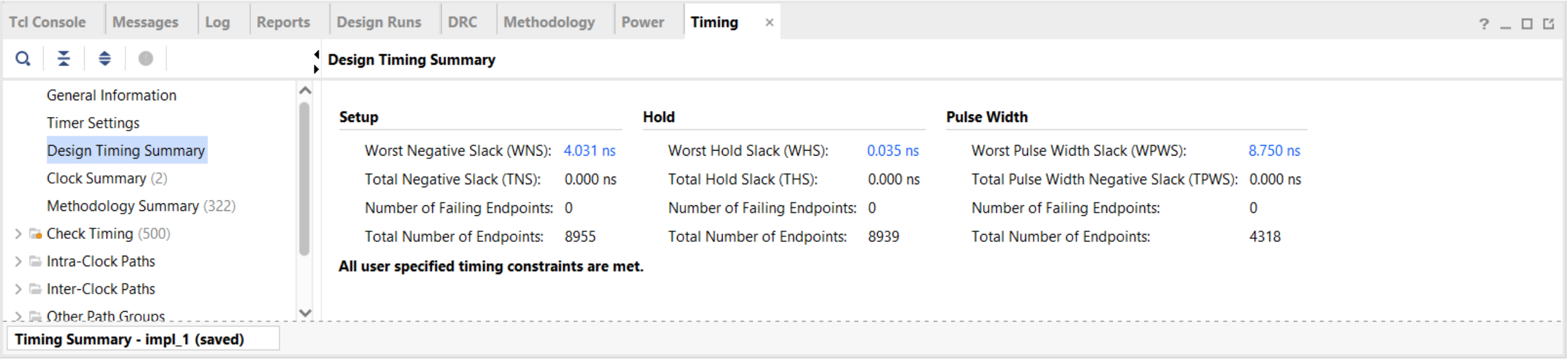
Launch Power Constraint Advisor to find and fix invalid switching activity



총 소모 에너지 = 전력 × 소요시간

- 버스 오버헤드를 제거하고 4 Cycle/MAC의 성능을 달성하여 연산 시간 단축
- 동일한 데이터량을 처리하는 데 필요한 전체 시스템 에너지를 근본적으로 절감
- 실제 선단 공정 테이프아웃 시, BEOL의 기생 커패시턴스 충방전에 의한 전력 낭비와 국부적 발열 문제를 아키텍처 레벨에서 사전 해결할 수 있는 물리적 설계

8. 결론 (타이밍 마진)



- PIM-SoC 아키텍처는 50MHz로 동작을 검증, Timing Summary 분석 결과 WNS 4.031ns를 확보하여 이론적으로 최대 약 62.6MHz까지 오버클럭이 가능한 안정적인 설계.
- 실리콘 양산 시 발생하는 PVT Variation 환경 속에서 타이밍 마진을 확보를 통해 견고한 물리적 설계 입증.

8. 결론 (자원사용량 - ILA 포트 5개 운용 시)

Tcl Console | Messages | Log | Reports | Design Runs | DRC | Methodology | Power | Timing | Utilization

Hierarchy
Summary
Slice Logic
Slice LUTs (12%)
LUT as Memory (2%)
LUT as Shift Regist
LUT as Distributed
LUT as Logic (11%)

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	Block RAM Tile (50)	Bonded IOB (250)	BUFGCTRL (32)	BSCANE2 (4)
FPGA	2411	2728	3	1079	2263	148	39.5	21	2	1
dbg_hub (dbg_hub)	448	741	0	240	424	24	0	0	1	1
u_cpu (cpu)	937	377	0	363	937	0	15	0	0	0
u_ssd (io)	12	20	0	13	12	0	0	0	0	0
wrap0 (uart_wrap)	55	48	0	22	55	0	0	0	0	0
your_ila (ila_0)	922	1484	3	450	798	124	24.5	0	0	0

utilization_1

Tcl Console | Messages | Log | Reports | Design Runs | DRC | Methodology | Power | Timing | Utilization

Hierarchy
Summary
Slice Logic
Slice LUTs (12%)
LUT as Memory (2%)
LUT as Shift Regist
LUT as Distributed
LUT as Logic (11%)

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	Block RAM Tile (50)	DSPs (90)	Bonded IOB (250)	BUFGCTRL (32)	BSCANE2 (4)
FPGA	2469	2745	3	1106	2321	148	39.5	1	21	2	1
dbg_hub (dbg_hub)	448	741	0	225	424	24	0	0	0	1	1
u_cpu (cpu)	995	394	0	374	995	0	15	1	0	0	0
u_ssd (io)	12	20	0	12	12	0	0	0	0	0	0
wrap0 (uart_wrap)	55	48	0	21	55	0	0	0	0	0	0
your_ila (ila_0)	922	1484	3	473	798	124	24.5	0	0	0	0

utilization_1

MAC 연산 추가 시
DSP 자원 추가

8. 결론 (자원사용량 – ILA 포트 5개 운용 시)

	전체	사용량
LUT	20800	2469(11.87%)
LUTRAM	9600	148(1.54%)
FF	41600	2713(6.52%)
BRAM	50	39.50(79%)
IO	250	21(8.40%)
BUFG	32	2(6.25%)
DSP	90	1(1.11%)

9. 설계의의

- **Custom CPU & 가속기(PIM) 설계**
  - 멀티사이클 기반의 CPU 명령어 셋 및 제어부 설계
  - 행렬 곱(MAC) 연산에 특화된 하드웨어 PIM 가속기 RTL 모델링
- **업계 표준 인터페이스(AMBA AXI4) 기반 시스템 통합**
  - 산업 표준 규격인 AXI4-Lite 버스를 채택하여 SoC 환경 구축
  - 이기종 IP 간의 통신 호환성 및 향후 확장성 기반 마련
- **비동기 통신을 통한 시스템 안정성 확보**
  - 독립된 채널 구조(VALID/READY)를 활용한 비동기 핸드쉐이크 구현
  - CPU와 PIM 간의 타이밍 불일치를 시스템 데드락 원천 차단

10. 발전가능성 및 고도화 전략

1

파이프라이닝 도입

- 5-Stage 구조 전환으로 해저드 제거

2

멀티 마스터 시스템 확장

- AXI4 Full 및 DMA 컨트롤러 탑재

3

산업계 연계 비전

- 로직과 메모리 다이를 TSV 및 하이브리드 본딩 기술로 3D 수직 적층 시, PIM 반도체 상용화 기대